Kingdom of Saudi Arabia
Ministry of Education
Prince Sattam Bin Abdulaziz University
Applied College

المملكة العربية السعودية
وزارة التعليم
جامعة الأمير سطام بن عبد العزيز
الكلية التطبيقية

A Smart Pharmacy System for Medicine Discovery, Real-Time Availability Tracking, and Efficient Order Management

A report submitted in partial fulfillment of the requirements for the Case study in Programming and Computer Science program – CASE 2611

by

Bayan Nasser Dhafyan Al-Dossari	***445954515***
Manal Majed Rasan Al-Dossari	***445954516***
Manal Mohammed Salman Al-Fifi	***445954510***
Jamila Nasser Saeed Al-Dossari	***445955360***
Alin Zaid Majed Al-Dossari	***445954532***
Noura Nasser Dhafyan Al-Dossari	***445954501***
Noura Abdullah Barhoum Al-Dossari	***445954409***
Layan Meshaal Ali Al-Dossari	***445954432***
Salma Saad Tanf Al-Dossari	***444951643***
Mayas Jaloud Mesfer Al-Dossari	***445954404***
Almas Mesfer Hamed Al-Ghamdi	***445954490***

Supervised by

DR./ Amal Hamdy

Assistant Professor - Computer Applications in the Science of Physics

2026 – Second Semester

1447/2026 © Applied College in wadi al-dwasir

Kingdom of Saudi Arabia
Ministry of Education
Prince Sattam Bin Abdulaziz University
Applied College

المملكة العربية السعودية
وزارة التعليم
جامعة الأمير سطام بن عبد العزيز
الكلية التطبيقية

نظام صيدليّة ذكي لاكتشاف الأدوية وتتبع توفرها في الوقت الحقيقي وإدارة الطلبات بكفاءة

تقرير دراسة حالة مقدم كمتطلب للحصول على درجة الدبلوم في برنامج البرمجة وعلوم الحاسب – حالة 2611

إعداد

بيان ناصر ضفيان الدوسري	***445954515***
منال ماجد راسان الدوسري	***445954516***
منال محمد سلمان الفيفي	***445954510***
جميله ناصر سعيد الدوسري	***445955360***
الين زيد ماجد الدوسري	***445954532***
نوره ناصر ضفيان الدوسري	***445954501***
نوره عبدالله برهوم الدوسري	***445954409***
ليان مشعل علي الدوسري	***445954432***
سلمى سعد طنف الدوسري	***444951643***
مياس جلعود مسفر الدوسري	***445954404***
الماس مسفر حامد الغامدي	***445954490***

إشراف

د/ أمل حمدي

استاذ مساعد – تطبيقات الحاسب الالي في علم الفيزياء

1447هـ – الفصل الثاني

2026/1447 © الكلية التطبيقية فرع وادي الدواسر

بسم الله الرحمن الرحیم

Contents

Certificate from the Supervisor		v
List of Figures		vi
List of tables		viii
Abstract		ix
CHAPTER 1: Introduction		
1-1	Introduction	1
1-2	Problem Statement	1
1-3	Project Overview	2
1-4	Objectives	2
1-5	Significance of the Project	3
1-6	Study Results	3
1-7	Survey	3
1-8	Previous Studies	10
1-9	Scope and Limitations	12
1-10	Case Study Requirement	12
1-11	Software Development Cycle	13
1-12	Conclusion	14
CHAPTER 2: System Analysis		
2-1	Introduction	15
2-2	System Specification	15
2-3	Use Case Diagram	17
2-4	Activity Diagrams	18
2-5	Entity Relationship Diagram (ERD)	22
2-6	Conclusion	23
CHAPTER 3: System Design		
3-1	Introduction	24
3-2	Class Diagram	24
3-3	Data Flow Diagrams	26
3-4	Sequence Diagrams	27
3-5	Conclusion	32
CHAPTER 4: System Implementation		
4-1	Introduction	33

4-2	Database Implementation	33
4-3	Frontend Implementation	34
4-4	Backend Implementation	42
4-5	Conclusion	47
CHAPTER 5: System Testing and Deployment		
5-1	Introduction	48
5-2	Testing Objectives	48
5-3	Testing Methodologies	49
5-4	Testing Tools	50
5-5	Testing Process	50
5-6	Testing Outcomes	51
5-7	System Deployment	51
5-8	Conclusion	52
CHAPTER 6: Conclusion and Future Works		
6-1	Conclusion	53
6-2	Future Works	53
References		55
Arabic Abstract		57

CERTIFICATE

This is to certify that the case study titled “A Smart Pharmacy System for Medicine Discovery, Real-Time Availability Tracking, and Efficient Order Management ” was undertaken by

Bayan Nasser Dhafyan Al-Dossari	***445954515***
Manal Majed Rasan Al-Dossari	***445954516***
Manal Mohammed Salman Al-Fifi	***445954510***
Jamila Nasser Saeed Al-Dossari	***445955360***
Alin Zaid Majed Al-Dossari	***445954532***
Noura Nasser Dhafyan Al-Dossari	***445954501***
Noura Abdullah Barhoum Al-Dossari	***445954409***
Layan Meshaal Ali Al-Dossari	***445954432***
Salma Saad Tanf Al-Dossari	***444951643***
Mayas Jaloud Mesfer Al-Dossari	***445954404***
Almas Mesfer Hamed Al-Ghamdi	***445954490***

All chapters were prepared by all of the above-mentioned students, they all cooperated in implementing the case study, and the consolidated report was prepared by all of them under my supervision.

Supervised by

DR./ Amal Hamdy

Assistant Professor - Computer Applications in the Science of Physics

2026 – Second Semester

List of Figures

Figure	**1-1**	User responses regarding difficulty in obtaining medicines	4
Figure	**1-2**	Main challenges faced when searching for medicines	5
Figure	**1-3**	User preference for using a pharmacy application	5
Figure	**1-4**	Most important features required in a pharmacy system	6
Figure	**1-5**	Importance of knowing medicine prices in advance	6
Figure	**1-6**	User preference for communicating with pharmacists through the application	7
Figure	**1-7**	User preference for WhatsApp as an alternative communication method	7
Figure	**1-8**	User need for medicine refill functionality	8
Figure	**1-9**	User opinion on saving previous medicines	8
Figure	**1-10**	User interest in a multi-pharmacy platform	9
Figure	**1-11**	User perception of time and effort savings provided by the system	9
Figure	**1-12**	Software Development Life Cycle (SDLC) Phases	14
Figure	**2-1**	Use Case Diagram	18
Figure	**2-2**	User Activity Diagram	19
Figure	**2-3**	Pharmacist Activity Diagram	20
Figure	**2-4**	Admin Activity Diagram	21
Figure	**2-5**	Entity Relationship Diagram	22
Figure	**3-1**	Class Diagram	25
Figure	**3-2**	DFD Level 0	26
Figure	**3-3**	DFD Level 1	27
Figure	**3-4**	User Registration and Login Sequence Diagram	28
Figure	**3-5**	Medicine Search, Cart, and Order Sequence Diagram	29
Figure	**3-6**	Pharmacist Add and Update Medicine Sequence Diagram	30
Figure	**3-7**	Reviews Prescription and Order Sequence Diagram	31

Figure	**4-1**		
Figure	**4-2**		
Figure	**4-3**		
Figure	**4-4**		
Figure	**4-5**		
Figure	**4-6**		
Figure	**4-7**		
Figure	**4-8**		
Figure	**4-9**		
Figure	**4-10**		
Figure	**4-11**		
Figure	**4-12**		
Figure	**4-13**		
Figure	**4-14**		
Figure	**4-15**		

List of Tables

Table	**1-1**	Hardware Requirments	12
Table	**1-2**	Software Requirments	12
Table	**2-1**	Functional Requirements	16
Table	**2-2**	Non-Functional Requirements	17

Abstract

This study presents a user-centered smart pharmacy platform designed to address the challenges associated with accessing medicines. The system provides a unified digital environment that connects multiple pharmacies, enabling users to search for medicines, compare prices, and verify real-time availability across nearby locations. In addition to core discovery features, the platform integrates a communication module that supports direct interaction between users and pharmacists through an in-app chat system for consultation and guidance. For cases requiring further assistance, the system offers escalation to external communication channels such as WhatsApp using the pharmacy's registered contact information, ensuring continuity of care. The platform also supports smart saving of user medication history and prescription management, allowing efficient refill of recurring medicines and enhancing treatment continuity. The system is implemented using modern web technologies, with Next.js for the frontend and Supabase for backend services including database management, authentication, and real-time data processing, ensuring scalability and performance. A structured database model and well-defined workflows support efficient management of users, pharmacies, medicines, orders, and communication processes. The study highlights the role of smart pharmacy systems in improving accessibility to medicines, enhancing user experience, and supporting digital transformation in healthcare services.

Keywords: Smart Pharmacy System, Medicine Discovery, Real-Time Availability, Pharmacist Interaction, Prescription Management, Patient-Centered Care.

Chapter 1: Introduction

1.1 Introduction

The rapid evolution of digital health technologies has significantly reshaped the delivery of healthcare services, particularly in improving patient access to medical resources and pharmaceutical care. Modern healthcare systems increasingly rely on digital platforms to enhance efficiency, accessibility, and patient engagement, enabling users to obtain services beyond traditional physical boundaries. In this context, telemedicine and digital health solutions have emerged as key enablers in transforming healthcare delivery by streamlining processes and facilitating timely access to services [1]. Within the pharmaceutical domain, the integration of such technologies has expanded the role of pharmacy services toward more patient-centered and technology-driven models, where digital tools such as mobile applications, real-time systems, and telehealth consultations contribute to improving patient outcomes and accessibility [2].

Building upon these advancements, smart pharmacy platforms represent an innovative approach that combines medicine discovery, real-time availability tracking, and interactive communication within a unified digital environment. These systems leverage modern web technologies and structured data models to connect users with multiple pharmacies, enabling efficient access to accurate information and supporting informed decision-making. Furthermore, the incorporation of communication features, including direct interaction with pharmacists and integration with external channels, enhances the continuity and quality of care. Consequently, the development of intelligent pharmacy systems aligns with the broader objectives of digital transformation in healthcare by improving service efficiency, reducing access barriers, and enhancing the overall user experience in pharmaceutical services.

1.2 Problem Statement

Patients frequently encounter inefficiencies when attempting to obtain medications due to the lack of a centralized system that provides reliable and up-to-date information on medicine availability and pricing across different pharmacies. The current reliance on physical visits or fragmented communication methods results in increased time, effort, and uncertainty, particularly when required medications are unavailable at nearby locations. In addition, the absence of integrated communication channels between patients and pharmacists limits access to professional guidance, which is essential for ensuring proper medication use and identifying suitable alternatives. Another significant challenge is the inability to effectively manage medication history and prescriptions, making it difficult for users to repeat previous treatments or maintain continuity of care. These limitations collectively highlight the need for a comprehensive digital solution capable of integrating pharmacy services, improving information accessibility, and supporting efficient interaction within the pharmaceutical care process.

1.3 Project Overview

The proposed system is a smart pharmacy platform that provides a unified digital environment for accessing and managing pharmaceutical services by connecting users with multiple pharmacies through a single interface. The platform enables efficient medicine discovery, real-time availability verification, and price comparison across different locations, thereby supporting informed decision-making and reducing the effort required to obtain medications. In addition to these core functionalities, the system integrates an interactive communication module that facilitates direct interaction between users and pharmacists through an in-application chat system for consultation and guidance, while also allowing escalation to external communication channels such as WhatsApp using the pharmacy's registered contact information when necessary. Furthermore, the system supports advanced medication management by enabling users to save frequently used medicines and manage prescriptions, which facilitates the efficient repetition of treatments and enhances continuity of care. The platform is implemented using modern web technologies to ensure scalability, real-time data processing, and reliable performance, while maintaining a user-friendly interface that improves accessibility and overall user experience, thereby providing a comprehensive digital solution to address key challenges in pharmaceutical service delivery.

1.4 Objectives

1. To design and implement a comprehensive smart pharmacy platform that provides a unified and user-centered digital environment for accessing pharmaceutical services across multiple pharmacies.
2. To enable efficient medicine discovery by providing accurate, real-time information on availability and pricing, along with comparison capabilities that support informed user decision-making.
3. To enhance the quality of interaction between users and pharmacists by integrating effective communication mechanisms, including in-application chat and the ability to extend communication through external channels when necessary.
4. To support continuity of care through advanced medication management features, including the ability to store user medication history, manage prescriptions, and facilitate the refill of recurring medicines.
5. To develop a scalable, reliable, and responsive system using modern web technologies that ensures high performance, usability, and accessibility across different devices and user needs.

1.5 Significance of the Project

The significance of this project lies in its ability to provide a practical and efficient solution that enhances the accessibility and quality of pharmaceutical services within a digitally integrated environment. By offering a centralized platform that connects multiple pharmacies, the system reduces reliance on traditional methods and minimizes the time and effort required to obtain accurate information about medications, thereby improving user convenience and supporting more efficient utilization of healthcare resources. In addition, the platform contributes to improving the reliability and organization of pharmaceutical service delivery by enabling consistent access to information and facilitating better coordination between users and pharmacists. The integration of communication and medication management features supports safer medication practices and promotes more informed decision-making, which is essential for improving overall healthcare outcomes. From a technological perspective, the project demonstrates the effective application of modern web-based solutions to address real-world challenges, highlighting the role of digital systems in enhancing efficiency, scalability, and service quality, while also contributing academically by presenting a model that can be extended to support future developments in digital healthcare.

1.6 Study Results

The findings of this study demonstrate that the proposed system achieves a high level of usability and functional efficiency in supporting pharmaceutical service processes within a digital environment. The evaluation indicates that users are able to interact with the system in a structured and intuitive manner, which contributes to improving task completion and reducing operational complexity. The integration of multiple functionalities within a single platform enhances system coherence and ensures seamless interaction between different components, particularly in relation to user interaction, data handling, and communication processes. In addition, the system exhibits stable performance and responsiveness under typical usage scenarios, reflecting the effectiveness of the adopted technologies in supporting real-time operations. These results highlight the system's capability to deliver a reliable and well-integrated solution that aligns with user requirements and supports the practical application of digital approaches in pharmaceutical service management.

1.7 Survey

In order to gain a comprehensive understanding of user needs, expectations, and challenges related to accessing pharmaceutical services, data was collected from 150 participants through a structured survey. The purpose of the survey was to evaluate the difficulties users face when obtaining medicines, identify the most important features expected in a smart pharmacy system, assess user preferences toward digital solutions, and determine key functional requirements for the proposed system. The

collected data played a significant role in guiding the design of the system, particularly in defining its core functionalities and ensuring alignment with real user needs.

1.7.1 Difficulty in Obtaining Medicines

Initially, participants were asked whether they experience difficulty in obtaining medicines. The results indicate that 22% of respondents reported frequent difficulty, while 47.3% experienced occasional difficulty, and 30.7% reported no difficulty. These findings reveal that a large proportion of users face challenges at least intermittently when trying to access medications. This suggests that existing methods are not fully efficient and highlights the necessity of a system that simplifies access and improves availability visibility.

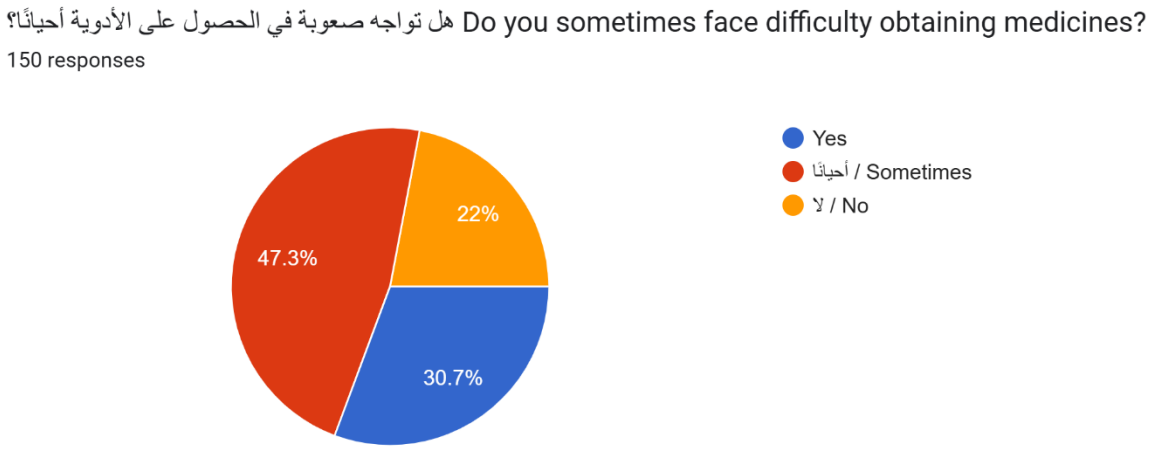

Figure 1-1: User responses regarding difficulty in obtaining medicines.

1.7.2 Main Challenges in Searching for Medicines

Participants were then asked to identify the main challenges they encounter when searching for medicines. The results show that the most significant issue is medicine unavailability (41.3%), followed by high prices (34.7%), and difficulty in identifying the appropriate pharmacy (23.3%). These findings indicate that users face multiple barriers simultaneously, including lack of information, cost concerns, and limited awareness of pharmacy locations. This clearly supports the need for a system that integrates availability tracking, price comparison, and multi-pharmacy access in a single platform.

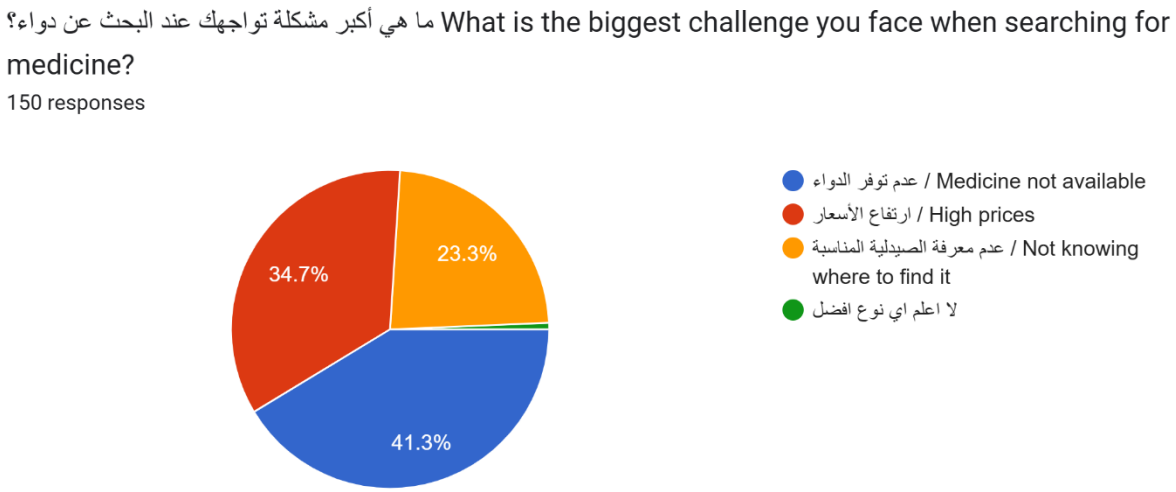

Figure 1-2: Main challenges faced when searching for medicines.

1.7.3 Preference for Using a Digital Application

When asked about their preference for using a digital application instead of physically visiting pharmacies, 71.3% of participants expressed a clear preference for using an application, while 19.3% were uncertain and only 9.3% preferred not to use one. These results demonstrate a strong acceptance of digital solutions among users and indicate readiness for adopting a smart pharmacy system.

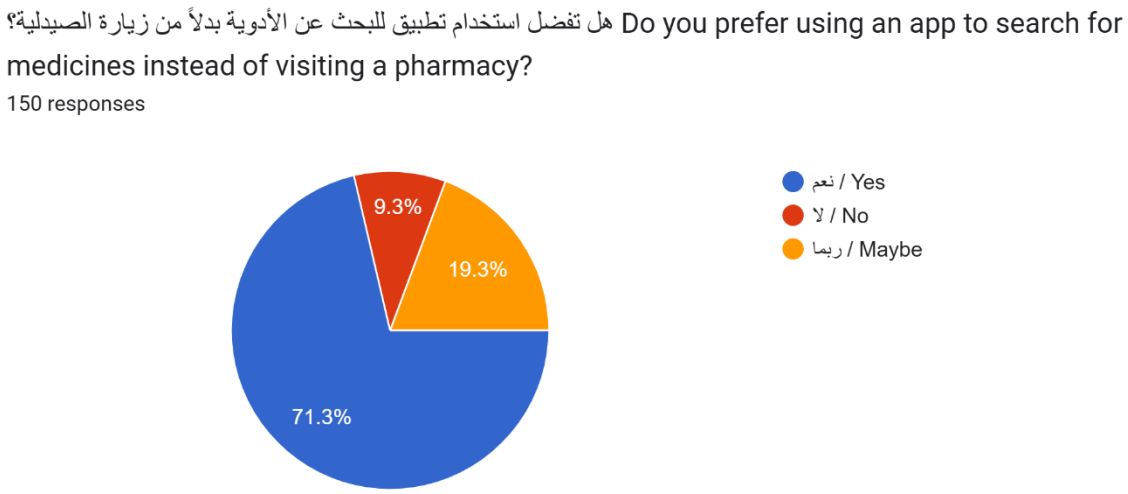

Figure 1-3: User preference for using a pharmacy application.

1.7.4 Most Important Features in a Pharmacy System

Participants were also asked to identify the most important feature they expect in a pharmacy application. The results indicate that price comparison was ranked highest (37.3%), followed by communication with pharmacists (32.7%), ordering medicines (18.7%), and checking availability (11.3%). This reflects that users prioritize transparency and interaction, rather than only basic search

functionalities, emphasizing the importance of integrating both informational and communication features in the system.

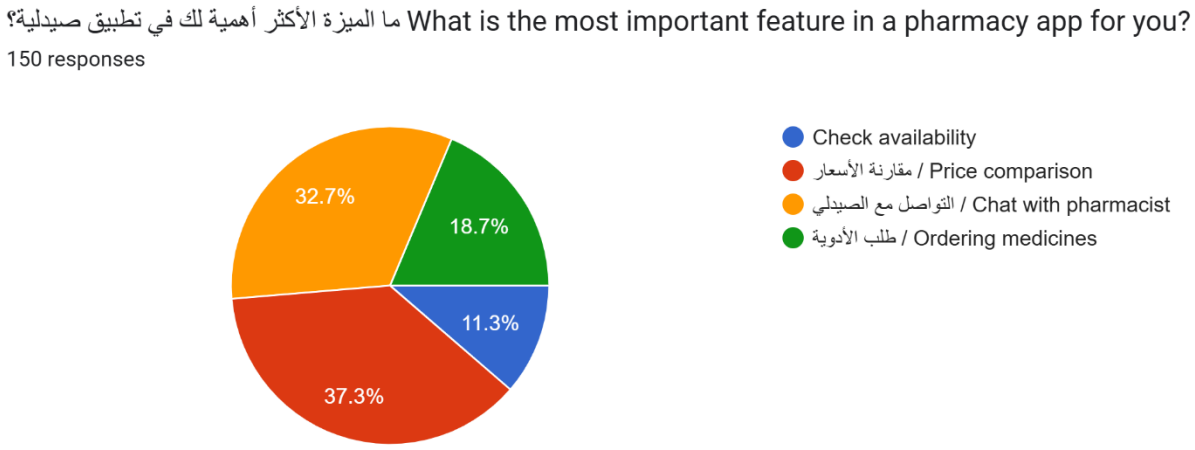

Figure 1-4: Most important features required in a pharmacy system.

1.7.5 Importance of Price Transparency

In response to the question regarding the importance of knowing medicine prices before visiting a pharmacy, 79.3% of participants indicated that it is very important, while 14.7% considered it important, and only 6% viewed it as not important. These results highlight the critical role of price transparency in improving user satisfaction and decision-making.

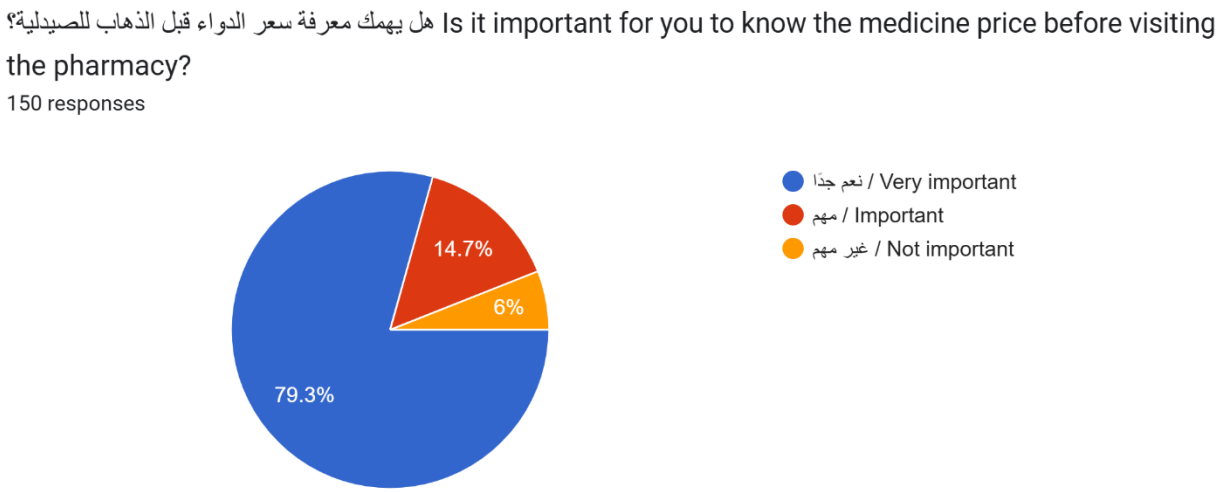

Figure 1-5: Importance of knowing medicine prices in advance.

1.7.6 Preference for Communication with Pharmacists

Participants were asked whether they prefer communicating with pharmacists through the application. The results show that 72% prefer this feature, 22% prefer it occasionally, and only 6% do not prefer

it. This demonstrates a strong demand for integrated communication tools that enable users to seek professional advice easily.

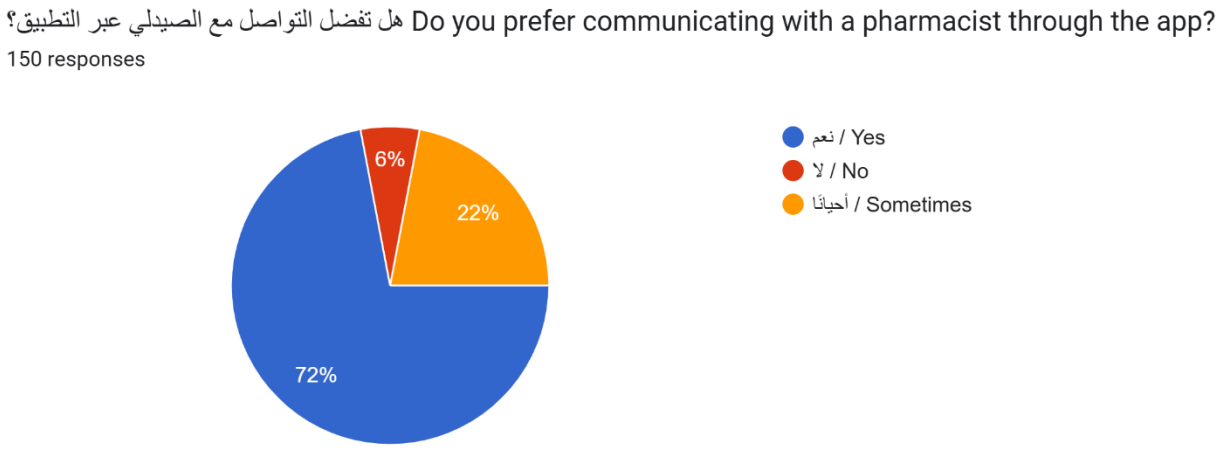

Figure 1-6: User preference for communicating with pharmacists through the application.

1.7.7 Preference for WhatsApp Communication

When evaluating the use of WhatsApp as an alternative communication channel, 87.3% of participants expressed preference for continuing communication through WhatsApp if in-app consultation is insufficient. This indicates that users value flexibility in communication and supports the integration of external communication channels within the system.

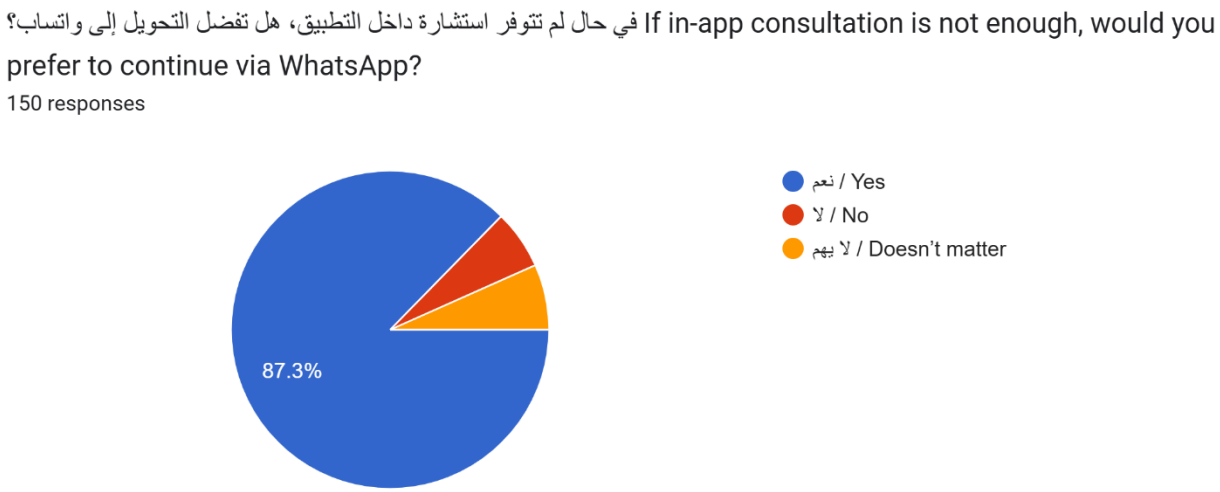

Figure 1-7: User preference for WhatsApp as an alternative communication method.

1.7.8 Need for Refill Functionality

The survey results reveal that 90% of participants have previously needed to reorder the same medicine, while only 10% have not. This demonstrates a strong need for implementing refill functionality, which can significantly improve user convenience and support continuity of treatment.

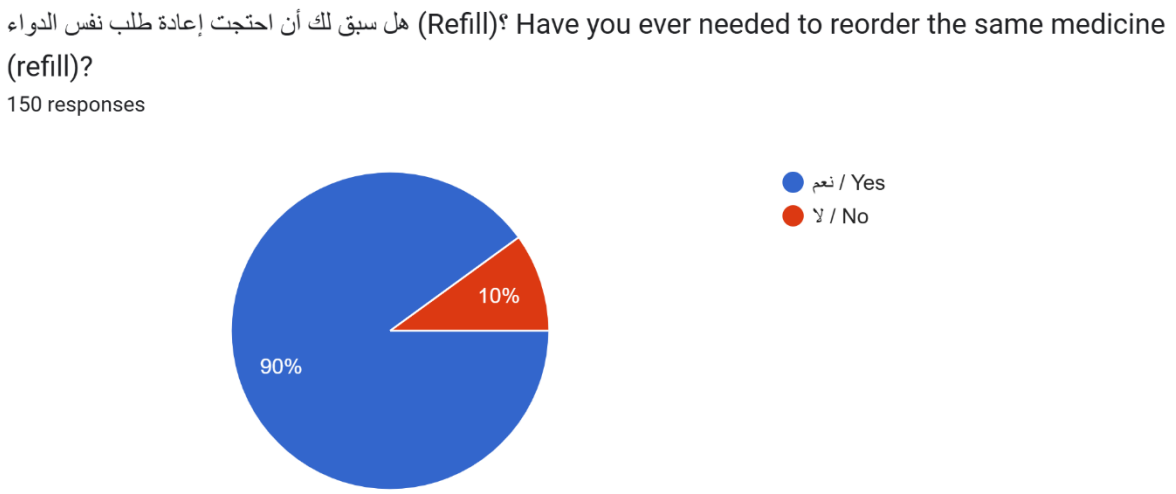

Figure 1-8: User need for medicine refill functionality.

1.7.9 Importance of Saving Medicines

Participants were asked whether saving previously used medicines would be helpful. The results show that 72% agreed, 22.7% were uncertain, and only a small percentage disagreed. These findings indicate that users value features that simplify repeated actions and support personal medication management.

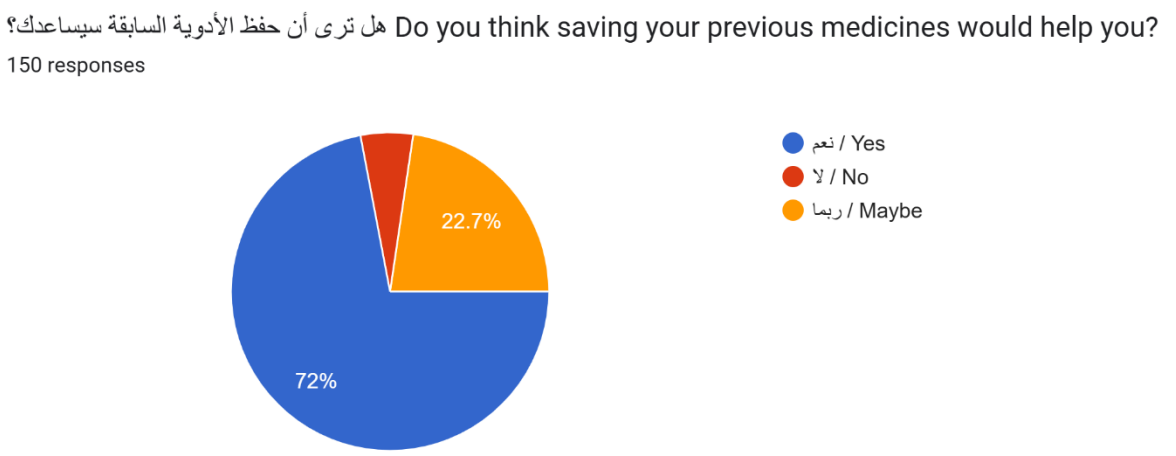

Figure 1-9: User opinion on saving previous medicines.

1.7.10 Interest in a Multi-Pharmacy Platform

A significant majority of participants (74.7%) expressed interest in using a platform that connects multiple pharmacies, while 24% showed very high interest and only 2 participants were not interested. This reflects strong acceptance of the system concept and its relevance to user needs.

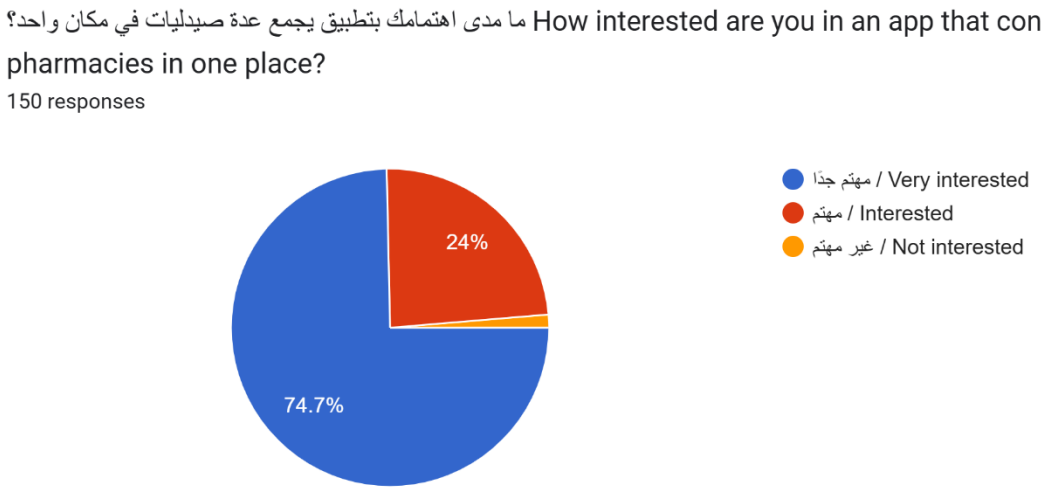

Figure 1-10: User interest in a multi-pharmacy platform.

1.7.11 Perceived System Benefits

Finally, participants were asked whether they believe the proposed system would save time and effort. The results show that 93.3% agreed, while only a very small percentage disagreed or were uncertain. This indicates a strong positive perception of the system's expected benefits and confirms its potential impact on improving efficiency in accessing pharmaceutical services.

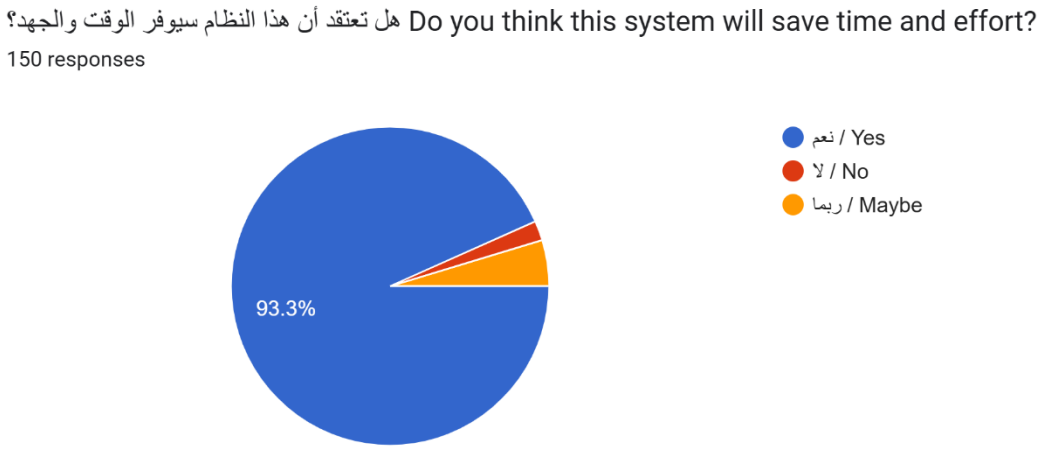

Figure 1-11: User perception of time and effort savings provided by the system.

1.8 Previous Studies

- **"Medic – The Medicine Availability Checker" by Aiswarya et al. (2025).**

This study presents a modern digital platform designed to improve patient access to medications by integrating pharmacy inventory systems into a unified solution. The system focuses on providing real-time information about medicine availability, pricing, and expiration dates across nearby pharmacies, allowing users to make informed decisions efficiently. The authors implemented advanced techniques such as location-based services using the Haversine formula to identify nearby pharmacies and cosine similarity algorithms to recommend alternative medicines based on composition. Additionally, the integration of an AI-powered chatbot enhances user interaction by providing automated responses to medicine-related queries. The study highlights that such systems significantly reduce the effort required to locate medications and improve healthcare accessibility. This research is particularly relevant as it demonstrates the importance of real-time data integration and intelligent assistance in pharmacy systems, which aligns closely with the objectives of the proposed smart pharmacy platform [3].

- **"MeDASH: An Effective Way to Check the Availability of Pharmaceutical Drugs in Proximity Using QR Codes" by Kommey et al. (2024).**

This research introduces a mobile-based solution that utilizes QR code technology to streamline the process of finding prescribed medications. The system allows doctors to generate QR-coded prescriptions, which patients can scan to locate pharmacies that have the required medicines in stock. The application integrates Google Maps for location tracking and employs efficient string-matching algorithms to enhance search performance. The results indicate that the system significantly reduces search time and improves user convenience by eliminating the need to visit multiple pharmacies physically. However, the study primarily focuses on prescription-based search and lacks features such as price comparison and direct communication with pharmacists. Despite these limitations, the study provides valuable insights into integrating prescription management with pharmacy systems, which can enhance the effectiveness of digital healthcare solutions [4].

- **"SmartMY: An Easy-Access Medication Price Checker Mobile Application Prototype" by Nik Mustaffa Shapri et al. (2025).**

This study focuses on improving price transparency in the pharmaceutical sector by developing a mobile application that provides users with access to official medication pricing data. The system integrates publicly available datasets from the Ministry of Health to allow users to search for medicines and view their prices based on generic or brand names. The findings show that users benefit from having clear pricing information before purchasing medications, which supports better financial

decision-making. However, the application relies on static datasets and does not provide real-time availability or multi-pharmacy comparison features. The study emphasizes the importance of price awareness in healthcare systems and highlights the need for integrating dynamic data sources to improve system functionality. This research contributes to the proposed system by reinforcing the necessity of including price comparison features alongside availability tracking [5].

- **"Design and Development of a Mobile Application for Drug Information and Other Health Data for Pharmacy Users" by Agudelo et al. (2025).**

This study adopts a user-centered design approach to develop a comprehensive mobile application aimed at enhancing access to pharmaceutical information and services. The system includes multiple functionalities such as drug information search, medication reminders, lifestyle guidance, and pharmacy geolocation. The research identified a large set of functional requirements and prioritized essential features to ensure usability and effectiveness. Evaluation results demonstrated high user satisfaction and successful task completion rates, indicating that the system significantly improves user experience and accessibility. The study highlights the importance of incorporating user needs into system design and emphasizes the role of usability in the success of healthcare applications. This work is particularly relevant as it supports the inclusion of features such as medication management and user-friendly interfaces in the proposed system [6].

- **"MediSync: A Smart Medicine Tracking and Pharmacy Integration System" by Adarsh et al. (2025).**

This research introduces an integrated system designed to enhance medication adherence and improve access to pharmacy services. The system provides automated SMS reminders for medication schedules and expiration dates while also connecting users to nearby pharmacies through real-time inventory updates. The implementation utilizes modern technologies such as React for the frontend, Python-based frameworks for the backend, and Firebase for real-time data synchronization. The results demonstrate that the system effectively supports users in managing their medications and reduces the risk of missed doses or expired drug usage. Additionally, the integration of real-time pharmacy data improves accessibility and convenience. This study is highly significant as it highlights the value of combining medication management with pharmacy integration, which directly supports the objectives of the proposed smart pharmacy system [7].

1.9 Scope and Limitations

The scope of this project involves the design and implementation of a smart pharmacy platform that enables medicine discovery, real-time availability checking, and price comparison across multiple pharmacies within a unified digital environment. The system supports user interaction through features such as account management, medication search, pharmacist communication, and basic medication management, aiming to improve accessibility and efficiency in pharmaceutical services. It is developed as a web-based application using modern technologies to ensure scalability, usability, and responsiveness.

However, the system has certain limitations. The accuracy of availability and pricing information depends on regular updates from pharmacies, which may affect data reliability. The platform also relies on internet connectivity, which may limit access in low-network environments. Additionally, integration with external communication channels is dependent on third-party services, and the system does not fully support end-to-end transaction processing. These constraints should be considered in future enhancements and real-world deployment.

1.10 Case Study Requirement

Component	**Specification**
Processor	Intel Core i5 / AMD Ryzen 5 or higher
Memory (RAM)	8 GB or more
Storage	512 GB SSD
Network	High-speed Internet connection
Server	Cloud-based server (Supabase infrastructure)

Table 1-1. Hardware Requirements.

Software Tool	**Version/Tool**
Operating System	Windows 10
Frontend Framework	Next.js (React)
Backend Platform	Supabase (Backend-as-a-Service)
Database	PostgreSQL (via Supabase)
Authentication	Supabase Auth
API	RESTful APIs / Supabase Client
IDE / Tools	Visual Studio Code

Table 1-2. Software Requirements.

1.11 Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured framework that guides software development through defined stages, ensuring alignment with user requirements, quality standards, and organizational objectives. By organizing the process into logical phases, it enhances planning, risk management, and consistency, while supporting the delivery of reliable and maintainable systems within controlled time and cost constraints [8].

1.11.1 How the SDLC Works

The SDLC operates through a sequential or iterative progression of phases that begins with identifying system requirements and extends to long-term system maintenance. Each phase generates specific outputs, such as requirement specifications, design models, or functional components, which serve as inputs for subsequent stages, ensuring a continuous and logical development flow. In addition, modern SDLC practices incorporate feedback mechanisms that allow revisiting earlier stages when necessary to address issues or refine system requirements. Different SDLC models, including Waterfall, Agile, and Spiral, vary in how these phases are organized, where some emphasize a linear progression while others support iterative refinement and adaptability. This structured approach provides a clear development roadmap and ensures alignment between technical implementation and stakeholder expectations throughout the project lifecycle [9].

1.11.2 Phases of SDLC

The SDLC is commonly organized into a set of core phases that collectively define the complete software development process [8]:

1. Requirement Analysis: This phase focuses on identifying and documenting both functional and non-functional requirements by gathering input from stakeholders, which defines the overall system scope and constraints.
2. System Design: In this stage, the collected requirements are transformed into technical specifications, including system architecture, database structures, and interface designs that guide implementation.
3. Implementation (Coding): The system is developed by writing and integrating code based on the design specifications, utilizing appropriate programming languages and development tools.
4. Testing: The developed system undergoes rigorous evaluation to verify its functionality, performance, and security, ensuring that it meets all specified requirements and operates without defects.

5. Deployment: The system is released into the operational environment, where it becomes accessible to end-users following proper configuration and setup.
6. Maintenance: This phase involves continuous monitoring, updates, and improvements to ensure system reliability, adaptability, and long-term performance.

These phases form a comprehensive framework that supports systematic software development, whether applied in a sequential manner, as in traditional models, or through iterative cycles in modern development approaches, thereby ensuring continuous alignment with evolving user needs and technological advancements [9].

Figure 1-12. Software Development Life Cycle (SDLC) Phases.

1.12 Conclusion

This chapter presented a comprehensive overview of the proposed smart pharmacy system, beginning with the contextual background and the challenges associated with accessing pharmaceutical services, and progressing through the definition of the problem, project objectives, and system scope. It also highlighted the significance of the system in improving accessibility, efficiency, and user experience within a digital healthcare environment. Furthermore, the analysis of survey results provided strong evidence of user needs and validated the relevance of the proposed solution, while the review of previous studies demonstrated the technological advancements and gaps addressed by this project. In addition, the chapter outlined the system requirements and the development approach based on the Software Development Life Cycle, establishing a structured foundation for the subsequent phases of system design and implementation.

Chapter 2: System Analysis

2.1 Introduction

System analysis represents a critical phase within the software development process, focusing on the systematic examination of user needs, operational challenges, and system requirements. It aims to transform abstract user expectations into clearly defined specifications that guide system design and implementation. Through this process, analysts evaluate existing workflows, identify inefficiencies, and determine both functional and non-functional requirements necessary for achieving system objectives, while also engaging with stakeholders to validate that system functionalities accurately reflect real-world needs. Moreover, system analysis establishes a structured understanding of system boundaries, data interactions, and operational constraints, which contributes to reducing development risks and improving overall system quality. It also supports the identification of appropriate system models and design approaches that facilitate efficient implementation. By applying formal modeling techniques and analytical methods, this phase ensures that the proposed system is aligned with user needs and organizational goals, thereby enhancing its effectiveness, reliability, and long-term sustainability [6].

2.2 System Specification

The system specification defines the functional capabilities and operational constraints of the proposed smart pharmacy platform, outlining how the system is expected to operate and interact with users. It serves as a structured representation of system requirements, ensuring clarity and consistency in development. This specification translates user needs into well-defined system behaviors, enabling developers to design and implement features that align with the intended objectives of the system. In addition, it establishes a common understanding among stakeholders regarding system functionalities, limitations, and performance expectations, which supports effective system development, testing, and future enhancements [10]. It also helps identify the main user roles, data flows, and system interactions required to deliver reliable pharmacy services. Therefore, the specification acts as a foundation for building a secure, usable, and scalable platform that meets both user and business needs.

2.2.1 Functional Requirements

Functional requirements delineate the specific operations, behaviors, and functions that must occur within a system. In other words, these specifications outline how the system will interact with a user, process data, run and synthesize organizational processes [10].

Functional Requirement	Description
User Registration & Authentication	The system shall allow users to create accounts, log in securely, and manage authentication to prevent unauthorized access.
User Role Management	The system shall support different user roles such as customer, pharmacist, and administrator with appropriate access permissions.
Medicine Search	The system shall enable users to search for medicines using names, categories, or keywords.
Real-Time Availability Check	The system shall display real-time availability of medicines across multiple pharmacies.
Price Comparison	The system shall allow users to compare medicine prices between different pharmacies.
Pharmacy Integration	The system shall connect multiple pharmacies within a unified platform for data sharing and service access.
Cart and Order Management	The system shall allow users to add medicines to a cart and place orders.
Pharmacist Communication	The system shall provide in-app chat functionality for users to communicate with pharmacists.
External Communication Support	The system shall allow communication extension through external platforms such as WhatsApp when needed.
Medication Management	The system shall allow users to save medicines and manage previous prescriptions for refill purposes.
Notification System	The system shall send notifications related to order updates, messages, and important system events.

Table 2-1. Functional Requirements.

2.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance constraints of the system, focusing on how the system operates rather than the specific services it provides. These requirements ensure that the system performs efficiently, securely, and reliably under different conditions [10].

Non-Functional Requirement	Description
Performance	The system shall provide fast response times for search, availability checks, and user interactions.
Usability	The system interface shall be user-friendly, intuitive, and accessible to users with different levels of technical experience.
Reliability	The system shall operate consistently with minimal downtime and ensure stable performance.
Security	The system shall protect user data through authentication mechanisms and secure data handling practices.
Scalability	The system shall support an increasing number of users and pharmacies without performance degradation.
Availability	The system shall be accessible online at all times with minimal service interruption.
Data Consistency	The system shall ensure accurate and synchronized data across all connected pharmacies.
Maintainability	The system shall be designed to allow easy updates, improvements, and bug fixes.

Table 2-2. Non-Functional Requirements.

2.3 Use Case Diagram

A use case diagram is a fundamental modeling tool used to represent the interaction between users and a system in terms of goals and functionalities. It focuses on identifying system services from the user's perspective by illustrating actors and their associated use cases. Rather than describing internal processes, the diagram emphasizes what the system is expected to accomplish, providing a high-level view of system functionality. This abstraction facilitates clear communication between stakeholders and developers, ensuring that system requirements are accurately captured and validated. Additionally, use case diagrams help in defining system boundaries and clarifying user roles, which supports the development of a consistent and user-centered design [11].

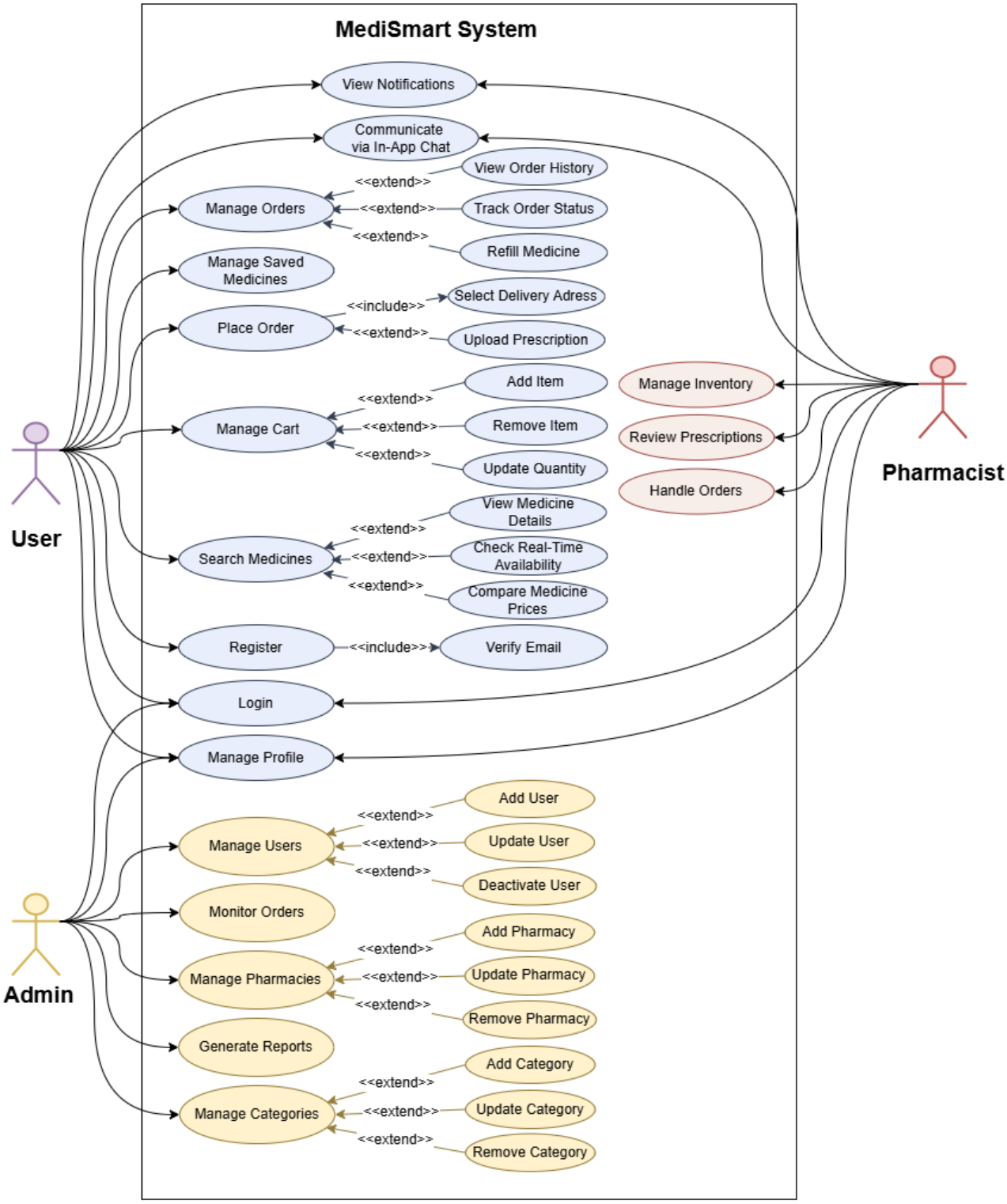

Figure 2-1: Use Case Diagram.

2.4 Activity Diagram

Activity diagrams are used to model the dynamic behavior of a system by representing workflows and sequences of operations. They illustrate how activities are executed, including decision points, parallel processes, and transitions between different states. These diagrams are particularly useful for describing complex processes in a simplified visual form, enabling a clear understanding of system

logic and operational flow. By capturing both sequential and concurrent activities, activity diagrams support detailed analysis of system behavior and assist in identifying potential inefficiencies or bottlenecks. Consequently, they play a significant role in validating system processes and improving clarity during the design phase [12].

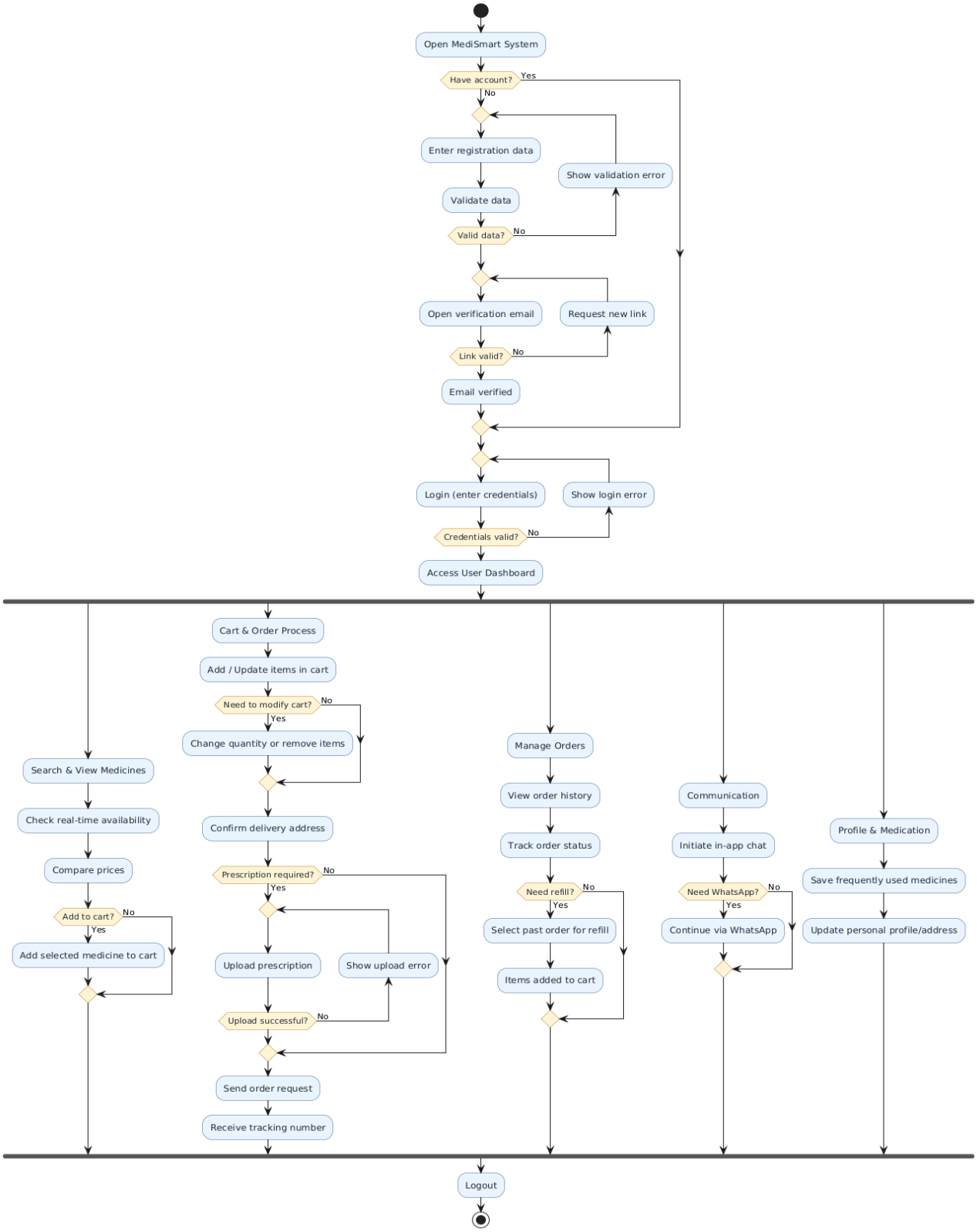

Figure 2-2: User Activity Diagram.

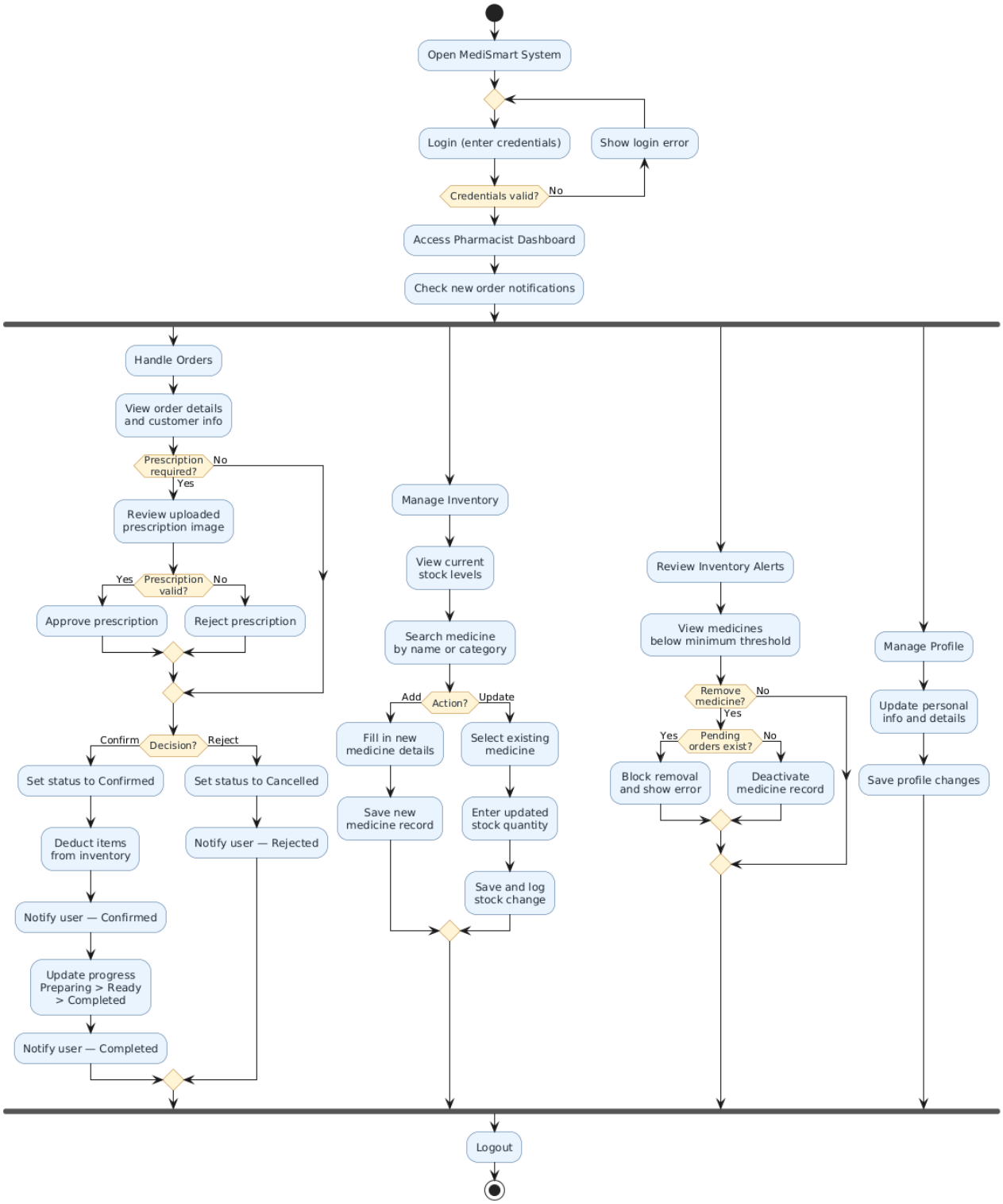

Figure 2-3: Pharmacist Activity Diagram.

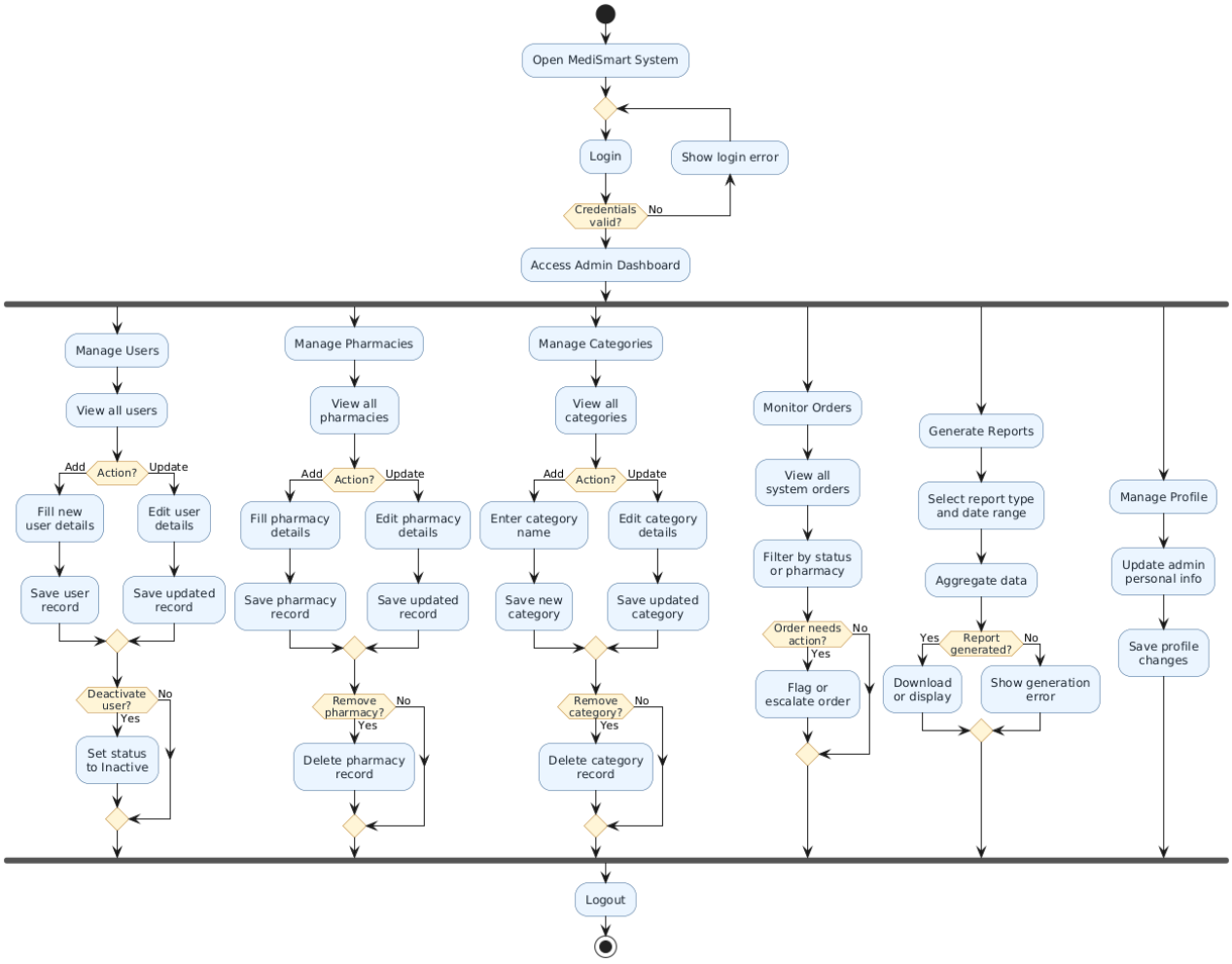

Figure 2-4: Admin Activity Diagram.

2.5 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) is a conceptual data modeling technique used to represent the structure of a database system. It illustrates entities, their attributes, and the relationships that connect them, providing a clear and organized view of the system's data requirements. By defining how data is stored, accessed, and related, the ERD serves as a foundation for designing a reliable and efficient database schema. It also ensures data consistency and integrity by clearly specifying relationships and constraints among entities. Furthermore, ERDs facilitate communication between analysts and developers by offering a visual representation of the data model, which simplifies the transition from conceptual design to physical database implementation [10].

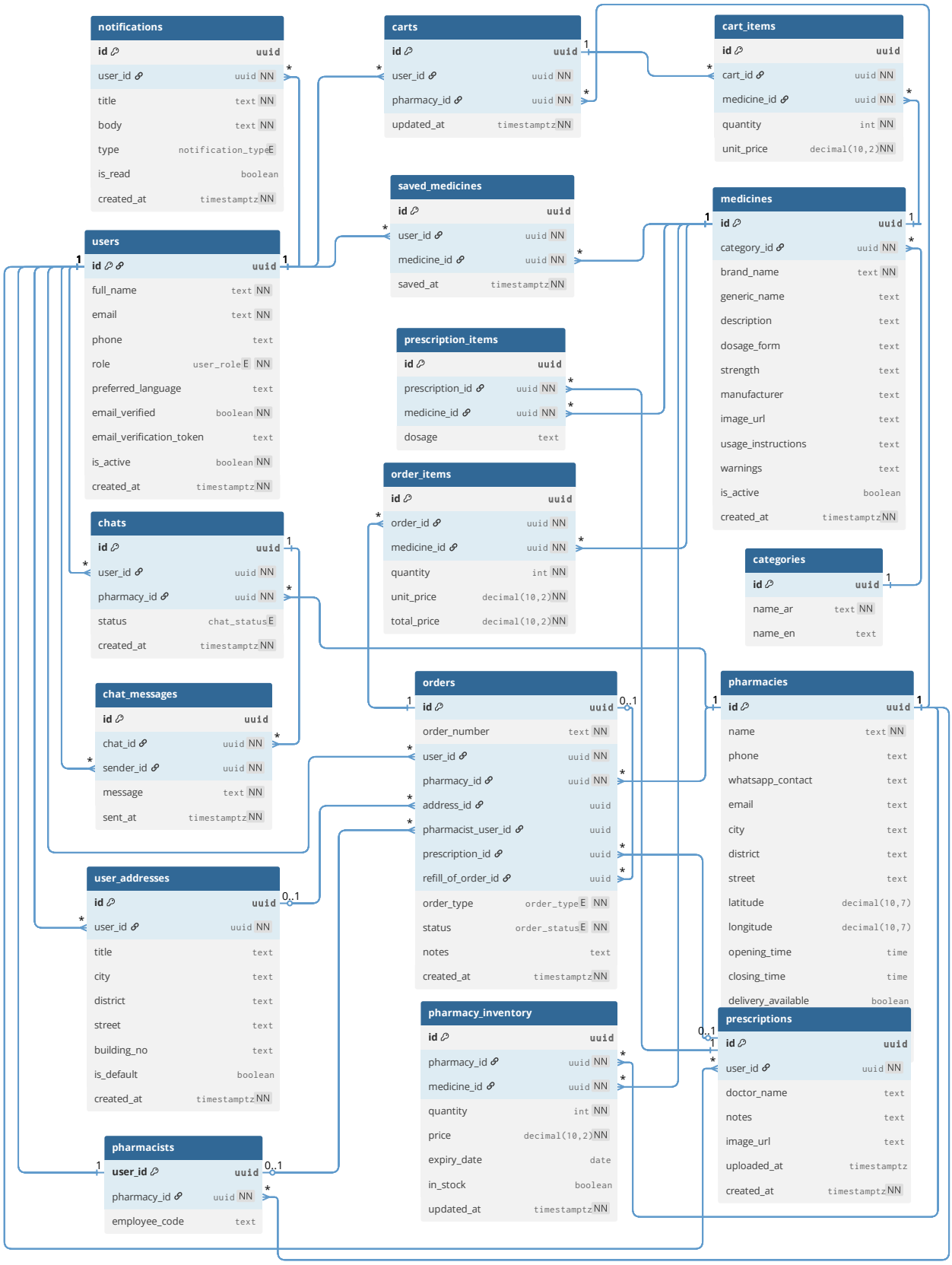

Figure 2-5. Entity Relationship Diagram.

2.6 Conclusion

This chapter presented a detailed analysis of the proposed smart pharmacy system, focusing on identifying system requirements and defining its core functionalities and structure. It outlined the system specifications, including both functional and non-functional requirements, which serve as a foundation for system design and implementation. In addition, the use of modeling techniques such as use case diagrams, activity diagrams, and the entity relationship diagram provided a clear representation of system behavior, user interactions, and data structure. These models contribute to improving understanding, reducing ambiguity, and ensuring consistency in system development. Overall, the system analysis phase establishes a solid basis for the subsequent design and implementation stages, ensuring that the system effectively meets user needs and achieves its intended objectives.

Chapter 3: System Design

3.1 Introduction

The system design phase represents the transition from analytical understanding to technical realization, where system requirements are transformed into a structured architectural model. This phase focuses on defining the organization of system components, data structures, and interaction mechanisms to ensure that all identified requirements are effectively implemented. Through a well-defined design approach, the system is organized into interrelated modules that collectively process inputs and produce meaningful outputs. In addition, system design establishes clear relationships between system elements, enabling efficient data handling and seamless communication between components. By employing standardized modeling techniques such as UML diagrams, the design process enhances clarity, reduces ambiguity, and supports collaboration among developers and stakeholders. Furthermore, a well-structured design ensures system scalability, maintainability, and adaptability to future enhancements, thereby contributing to the development of a robust and efficient software solution [13].

3.2 Class Diagram

The class diagram provides a static representation of the system structure by defining the main entities, their attributes, operations, and the relationships between them. In the proposed smart pharmacy system, core classes such as User, Pharmacy, Medicine, Cart, Order, and Chat are modeled to reflect real-world entities and their interactions within the system. Each class encapsulates both data and behavior, ensuring modularity and separation of concerns. Relationships such as association, aggregation, and composition are used to represent dependencies and ownership between different components, enabling a clear understanding of how system elements interact. This diagram plays a crucial role in translating conceptual requirements into a structured object-oriented design, ensuring data consistency and supporting efficient system implementation. Moreover, it serves as a foundational blueprint that guides developers during coding and integration phases [14].

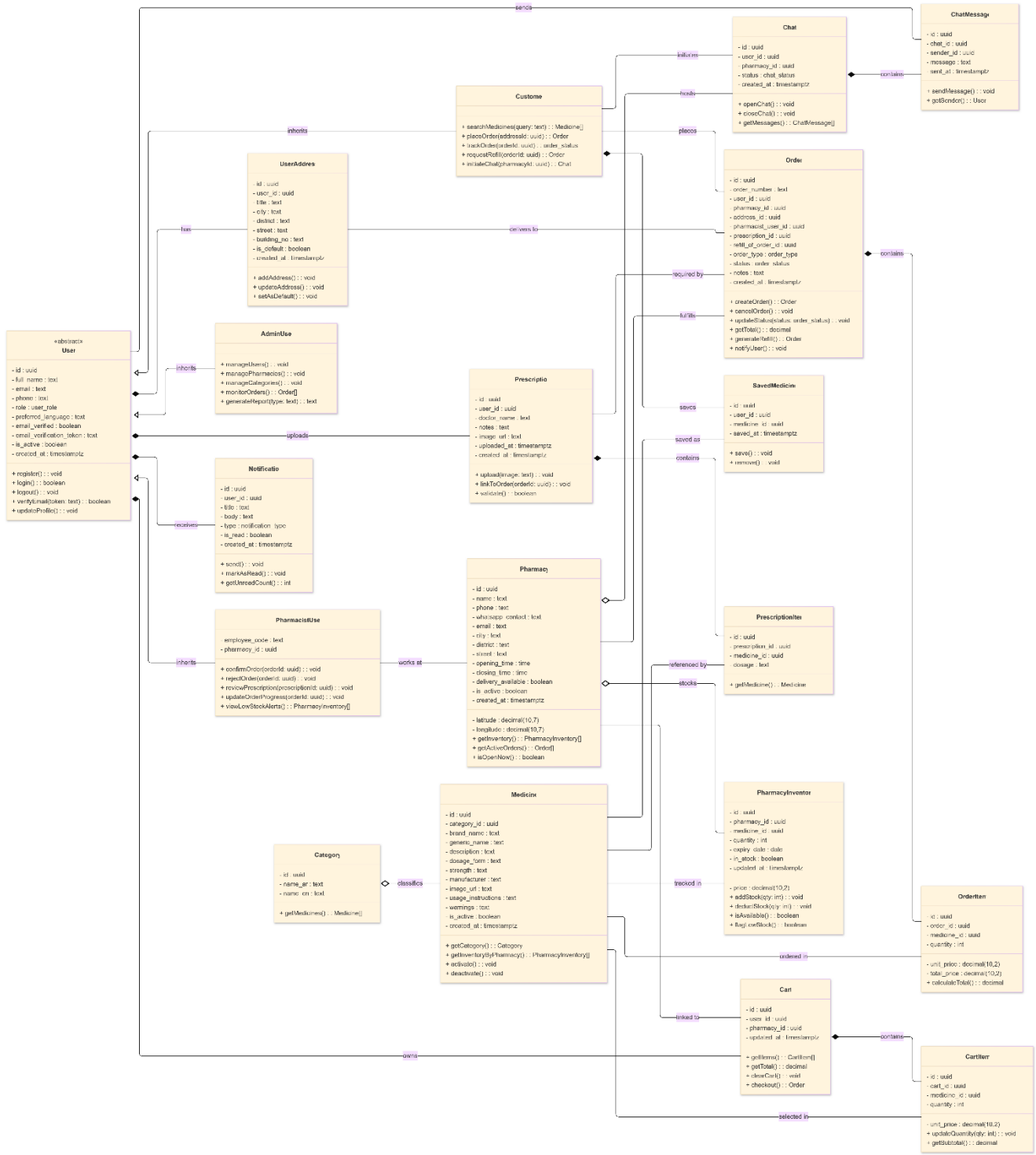

Figure 3-1: Class Diagram.

3.3 Data Flow Diagrams (DFDs)

Data Flow Diagrams (DFDs) are used to represent the logical movement and transformation of data within the system, illustrating how information flows between users, processes, and data stores. The Level 0 diagram, known as the context diagram, provides a high-level abstraction by modeling the entire system as a single process that interacts with external entities such as users and pharmacies, highlighting only the primary inputs and outputs without detailing internal operations. This overview is further refined in the Level 1 diagram, where the system is decomposed into multiple subprocesses representing key functionalities such as user authentication, medicine search, order management, and communication with pharmacists, with each process interacting with underlying data stores to retrieve and update information. By emphasizing data movement rather than implementation specifics, DFDs facilitate the identification of inefficiencies, reduce redundancy, and ensure consistency in data handling across the system, thereby enhancing the clarity of system operations and supporting effective design and validation [15].

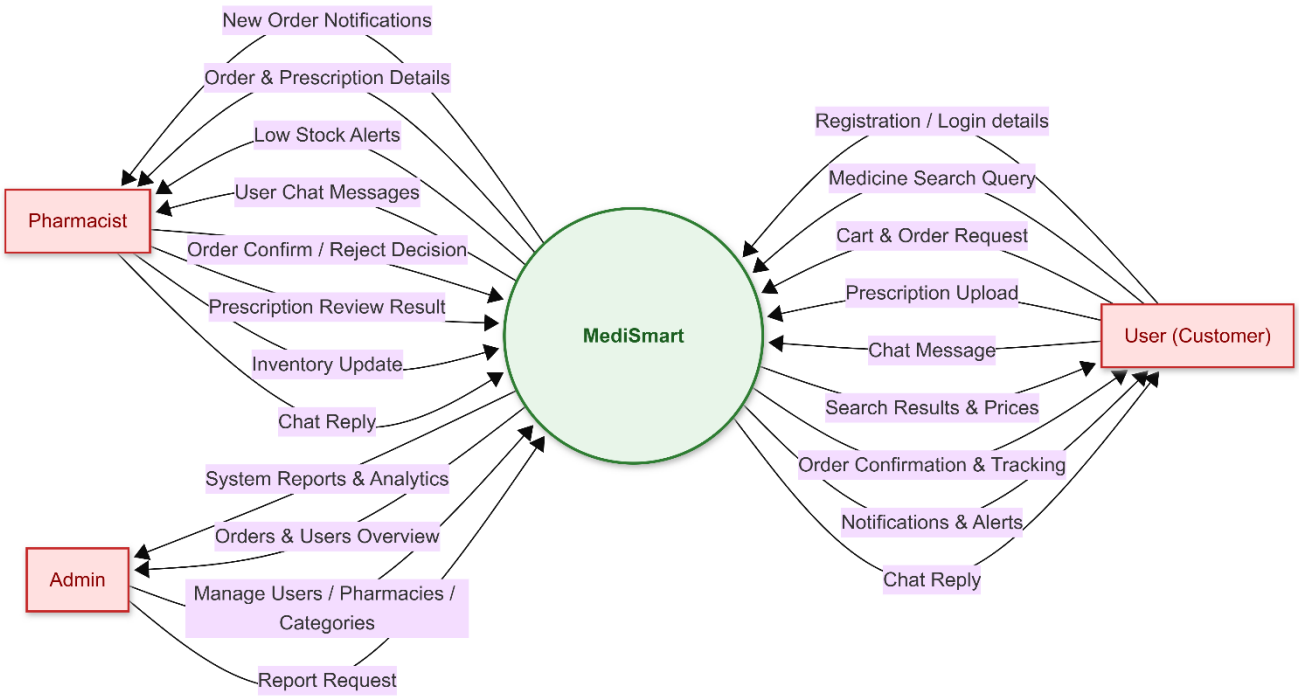

Figure 3-2. DFD Level 0.

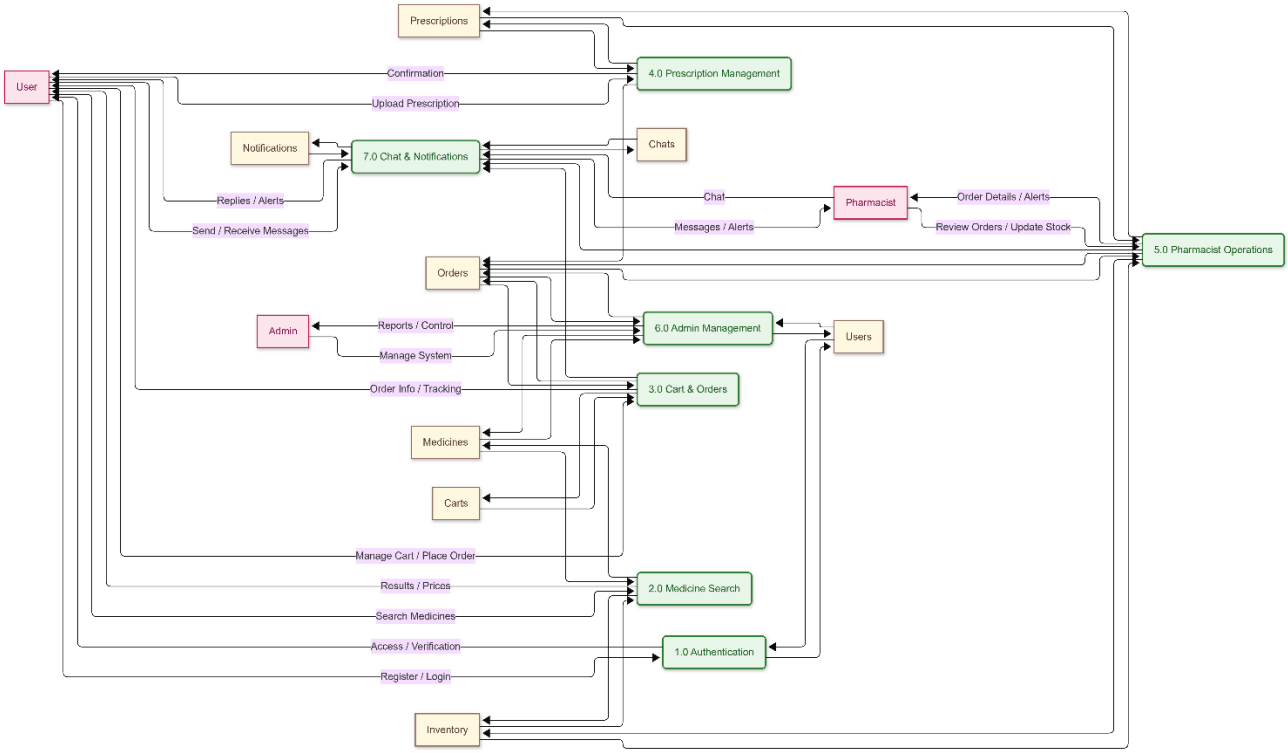

Figure 3-3: DFD Level 1.

3.4 Sequence Diagrams

Sequence diagrams provide a dynamic representation of system behavior by illustrating the time-ordered interactions between system components required to execute specific functionalities. These diagrams focus on how objects communicate through message exchanges to accomplish tasks such as user registration, medicine search, order placement, and interaction with pharmacists within the smart pharmacy system. Each component is represented as a lifeline, while messages are depicted in chronological order, clearly showing the sequence of operations and dependencies between system elements. This visualization enhances understanding of runtime behavior by demonstrating how different modules collaborate to achieve system objectives, while also helping to identify potential synchronization issues and ensuring that system interactions remain consistent with defined functional requirements and overall design structure [16].

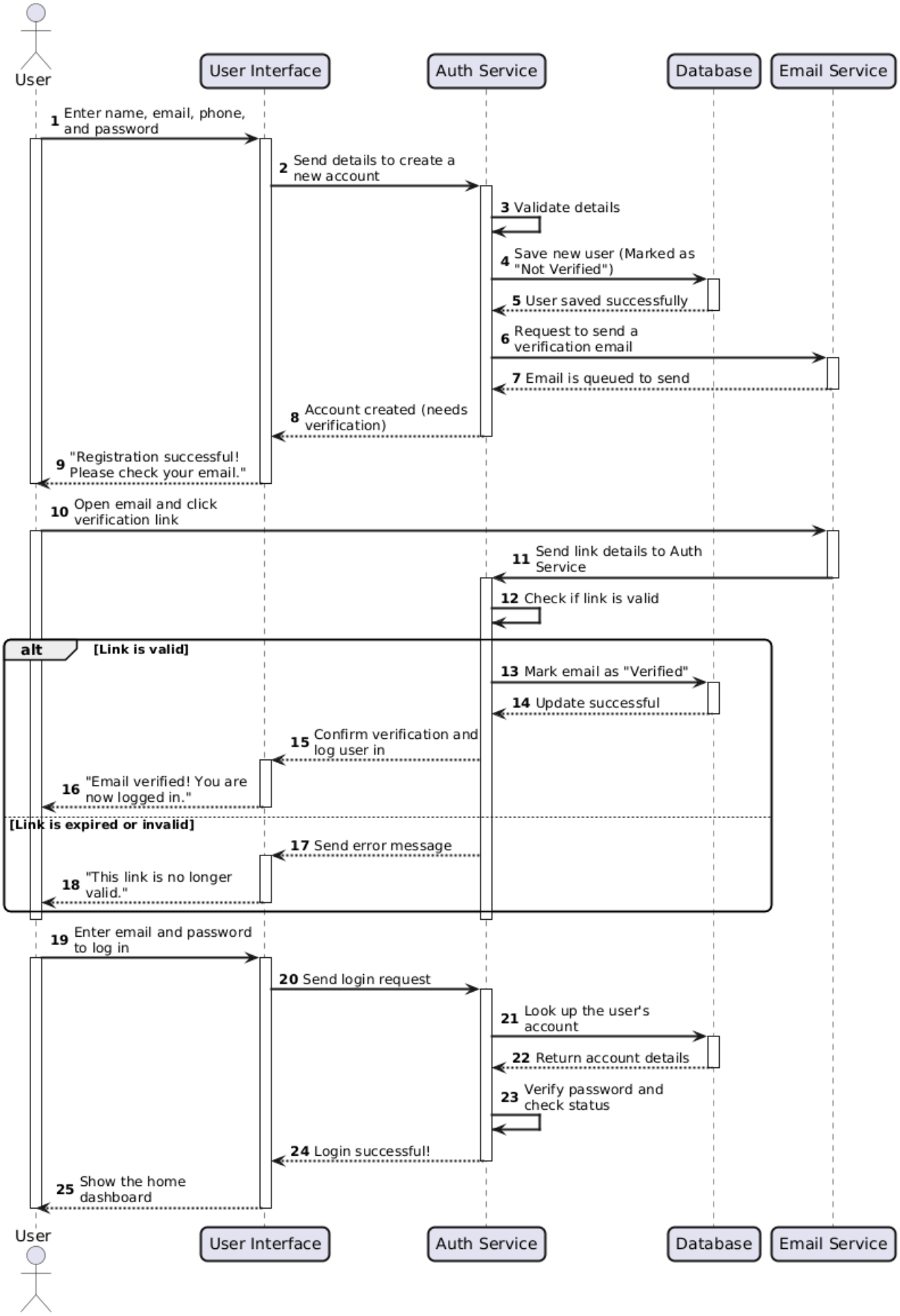

Figure 3-4: User Registration and Login Sequence Diagram.

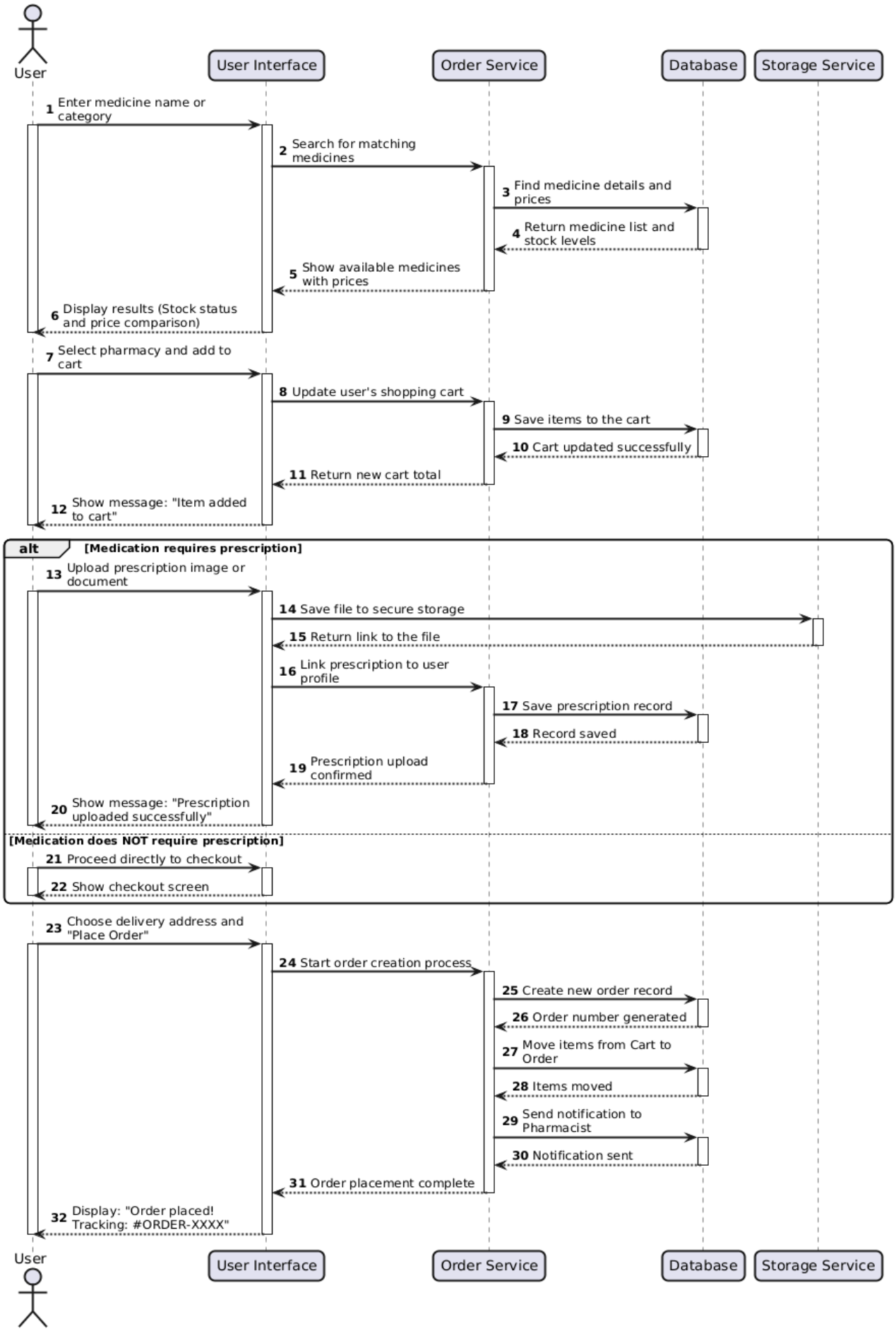

Figure 3-5: Medicine Search, Cart, and Order Sequence Diagram.

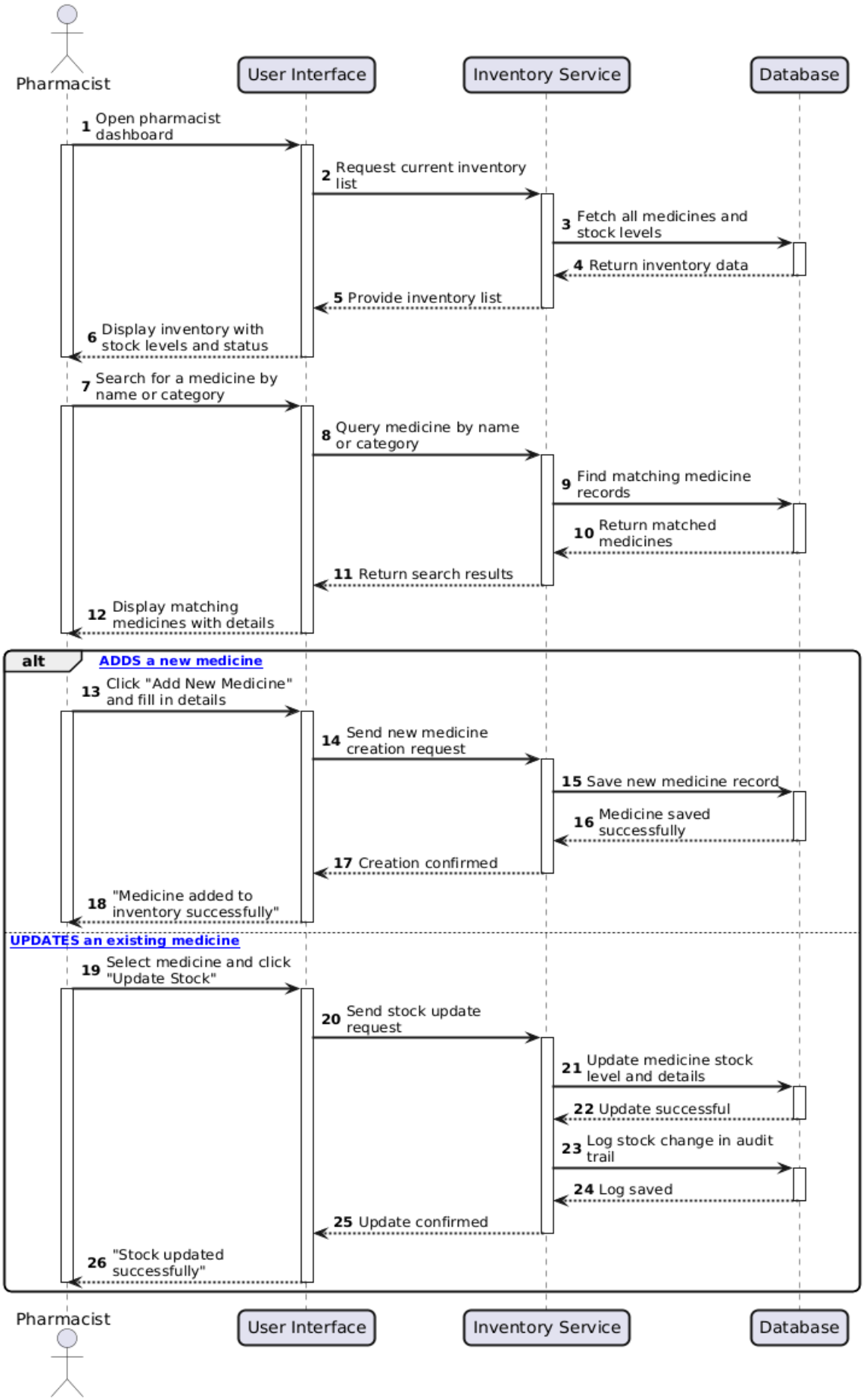

Figure 3-6: Pharmacist Add and Update Medicine Sequence Diagram.

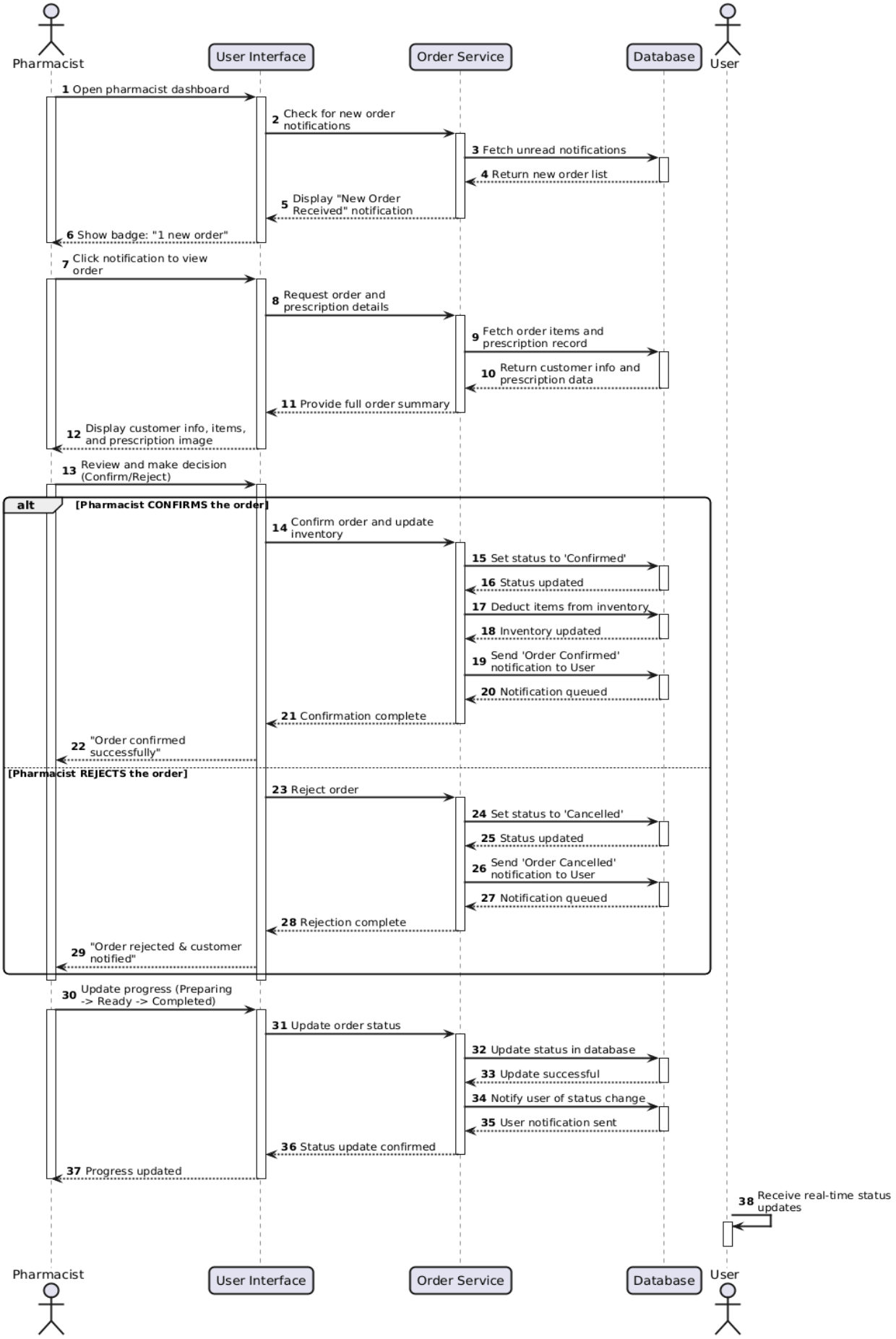

Figure 3-7: Reviews Prescription and Order Sequence Diagram.

3.5 Conclusion

This chapter presented the system design of the proposed smart pharmacy platform by translating analytical requirements into a structured technical framework. It outlined the system architecture through class diagrams, data flow diagrams, and sequence diagrams, which collectively describe the system structure, data movement, and dynamic interactions between components. These design models ensure clarity, consistency, and alignment with system requirements, while supporting scalability, maintainability, and efficient implementation. Overall, the system design establishes a solid foundation for the development phase and ensures that the system can effectively meet its intended objectives.

Chapter 4: System Implementation

4.1 Introduction

System implementation is the phase in which the MediSmart system design is converted into a functional web-based application. This chapter explains how the main system components were developed and connected, including the database, frontend interfaces, backend services, authentication, file storage, and system integration. The implementation was based on the requirements, use case diagram, activity diagrams, ERD, DFD, and sequence diagrams presented in the previous chapters. The system was developed using Next.js/React for the frontend and Supabase for backend services, including PostgreSQL database, authentication, storage, and real-time data processing. Through this implementation, MediSmart supports its main functions such as user registration and login, medicine searching, real-time availability checking, price comparison, cart and order management, prescription upload, pharmacist order review, inventory management, notifications, admin management, and report generation. This chapter therefore shows how the theoretical design of the smart pharmacy system was transformed into a practical digital platform that serves users, pharmacists, and administrators efficiently.

4.2 Database Implementation

The database infrastructure of the MediSmart system serves as the foundation of the platform, providing an organized framework for storing, retrieving, and managing data related to users, pharmacies, medicines, inventory, prescriptions, carts, orders, chats, and notifications. The database was implemented using PostgreSQL through Supabase, which provides a cloud-based environment suitable for scalable and real-time web applications. The schema was designed based on the Entity Relationship Diagram presented in the previous chapter, ensuring that all main system entities and relationships are represented clearly and consistently. The main tables include users, user_addresses, pharmacies, pharmacists, categories, medicines, pharmacy_inventory, carts, cart_items, orders, order_items, prescriptions, prescription_items, chats, chat_messages, notifications, and saved_medicines. Each table uses primary keys, mostly UUID values, while foreign key relationships connect related data such as users with orders, pharmacies with inventory, medicines with categories, and prescriptions with users.

The database design follows relational database principles to reduce redundancy, improve data consistency, and support reliable system operations. Enum types such as user_role, order_type, order_status, chat_status, and notification_type were used to control fixed values and organize important system states, such as user roles and order progress. Timestamp fields such as created_at, updated_at, uploaded_at, and sent_at were included to support auditing and system tracking. To

improve performance, indexes can be applied to frequently used fields such as user_id, pharmacy_id, medicine_id, category_id, email, order_status, and in_stock, which helps speed up medicine search, availability checking, order tracking, and inventory management. Through Supabase integration, the database supports real-time updates, authentication connection, file storage references, and efficient communication between the frontend and backend services.

4.3 Frontend Development

Frontend development is a central component of the system implementation phase, as it is responsible for constructing the visual and interactive elements through which users, pharmacists, and administrators interact with the MediSmart smart pharmacy system. In this project, the frontend layer was developed using Next.js with React, which provides a modern component-based structure, responsive page rendering, smooth navigation, and effective integration with backend services. This technology supports the development of reusable interface components such as forms, cards, tables, navigation bars, dashboards, and action buttons, which helps maintain consistency across the whole system.

The frontend was designed to be simple, clear, and user-friendly in order to support different types of users with different roles and technical abilities. The interfaces were connected with Supabase to retrieve and update data related to users, medicines, pharmacies, inventory, prescriptions, orders, chats, and notifications. Through this integration, the system can display real-time medicine availability, updated prices, order status, and pharmacist responses. The frontend implementation follows the workflows described in the use case diagram, activity diagrams, and sequence diagrams, ensuring that each interface supports the required functions of the system.

- **Frontend Code Sample for Medicine Search Interface:**

```
'use client';

import { useEffect, useMemo, useState } from 'react';
import { supabase } from '@/lib/supabase';
import { formatCurrency } from '@/lib/helpers';

export default function MedicinesPage() {
  const [query, setQuery] = useState('');
  const [categoryId, setCategoryId] = useState('');
  const [categories, setCategories] = useState<any[]>([]);
  const [rows, setRows] = useState<any[]>([]);
  const [message, setMessage] = useState('');

  useEffect(() => {
    async function loadMedicines() {
      const [{ data: cats }, { data: inventory }] = await Promise.all([
        supabase.from('categories').select('*').order('name_en'),
```

```
      supabase
        .from('pharmacy_inventory')
        .select('*, medicines(*, categories(*)), pharmacies(*)')
        .eq('in_stock', true)
        .gt('quantity', 0)
        .order('price', { ascending: true })
    ]);

    setCategories(cats || []);
    setRows(inventory || []);
  }

  loadMedicines();
}, []);

const filteredMedicines = useMemo(() => {
  const lower = query.toLowerCase();

  return rows.filter((row) => {
    const medicine = row.medicines;

    const matchesSearch =
      !lower ||
      [
        medicine?.brand_name,
        medicine?.generic_name,
        medicine?.description,
        medicine?.manufacturer
      ].some((value) => value?.toLowerCase().includes(lower));

    const matchesCategory =
      !categoryId || medicine?.category_id === categoryId;

    return matchesSearch && matchesCategory;
  });
}, [rows, query, categoryId]);

async function addToCart(row: any) {
  const { data: cart } = await supabase
    .from('carts')
    .select('*')
    .eq('pharmacy_id', row.pharmacy_id)
    .maybeSingle();

  await supabase.from('cart_items').insert({
    cart_id: cart?.id,
    medicine_id: row.medicine_id,
    quantity: 1,
    unit_price: row.price
  });
```

```
    setMessage('Medicine added to cart.');
  }

  return (
    <div>
      <h1>Medicine Search</h1>

      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
        placeholder="Search by brand, generic name, or manufacturer"
      />

      <select
        value={categoryId}
        onChange={(e) => setCategoryId(e.target.value)}
      >
        <option value="">All categories</option>
        {categories.map((category) => (
          <option key={category.id} value={category.id}>
            {category.name_en || category.name_ar}
          </option>
        ))}
      </select>

      {message && <p>{message}</p>}

      {filteredMedicines.map((row) => (
        <div key={row.id}>
          <h3>{row.medicines?.brand_name}</h3>
          <p>{row.medicines?.generic_name}</p>
          <p>Pharmacy: {row.pharmacies?.name}</p>
          <p>Stock: {row.quantity}</p>
          <p>Price: {formatCurrency(row.price)}</p>

          <button onClick={() => addToCart(row)}>
            Add to Cart
          </button>
        </div>
      ))}
    </div>
  );
}
```

This code represents the medicine search interface in the MediSmart system. It retrieves medicine inventory data from Supabase, displays available medicines with pharmacy and price details, filters results by keyword and category, and allows users to add selected medicines to the cart. This sample reflects the frontend role in supporting medicine discovery, availability checking, and price comparison.

4.3.1 User Interface

The User Interface represents the operational layer through which customers, pharmacists, and administrators use the MediSmart system. The interfaces were designed with a consistent layout, clear typography, organized sections, and accessible buttons to make navigation easy and efficient. This section provides a description of the main interfaces implemented in the system.

- **Authentication Interfaces**

The authentication interfaces include the registration, login, and account access pages. These interfaces allow users to create a new account or access an existing account securely. The registration page collects essential information such as full name, email, phone number, and password, while the login page verifies user credentials before allowing access to the system.

These pages were designed with clear input fields, validation messages, and action buttons to guide the user during authentication. After successful login, the system identifies the user role and redirects the user to the correct dashboard, whether the user is a customer, pharmacist, or administrator. This supports secure access and ensures that each actor can only use the functions related to their role.

- **User Homepage Interface**

The user homepage is the first main interface displayed after the customer logs into the system. It provides an overview of the most important services available in the MediSmart platform, such as medicine search, categories, nearby pharmacies, saved medicines, orders, notifications, and communication with pharmacists.

The homepage was designed to help users quickly access the main features without confusion. It includes navigation elements, search access, service cards, and highlighted sections that guide users toward common tasks such as searching for a medicine, checking availability, or reviewing previous orders. This interface supports the user-centered nature of the system by making medicine discovery faster and easier.

- **Medicine Search Interface**

The medicine search interface allows users to search for medicines using medicine names, generic names, categories, or keywords. This interface represents one of the most important functions of the system because it helps users find required medicines without visiting multiple pharmacies physically.

Search results are displayed in an organized layout that may include medicine name, dosage form, strength, category, image, and basic details. The interface also allows users to continue to the medicine details page to view more information. This design improves accessibility and supports quick medicine discovery.

- **Medicine Details and Availability Interface**

The medicine details interface provides detailed information about a selected medicine. It displays information such as brand name, generic name, description, dosage form, strength, manufacturer, usage instructions, warnings, and image. This helps users understand the medicine before adding it to the cart or contacting the pharmacist.

This interface also shows medicine availability across pharmacies, including price, stock status, and pharmacy information. By presenting availability and price information clearly, the system supports informed decision-making and reduces the time and effort needed to search for medicines manually.

- **Cart Interface**

The cart interface allows users to review the medicines they selected before placing an order. It displays the medicine name, quantity, unit price, total price, and selected pharmacy. Users can update quantities, remove items, or continue shopping before confirming the order.

This interface is important because it gives users a final opportunity to check their order details before submission. The cart design focuses on clarity and accuracy by organizing items in a readable format and providing clear buttons for updating, removing, and proceeding to checkout.

- **Prescription Upload Interface**

The prescription upload interface is used when a medicine or order requires a prescription. Through this page, the user can upload an image or file of the prescription and attach it to the order. The interface includes upload controls, prescription notes, and confirmation options.

This feature supports safe medication practices by allowing pharmacists to review prescription information before approving the order. The interface is designed to be simple so that users can upload prescriptions easily and complete the order process without difficulty.

- **Order Confirmation and Tracking Interface**

After placing an order, the order confirmation interface displays important order details such as order number, selected pharmacy, order type, total amount, and current status. This confirms that the order was submitted successfully and gives the user a clear summary of the transaction.

The order tracking interface allows users to follow the order progress through different statuses such as pending, confirmed, preparing, ready, out for delivery, completed, or cancelled. This interface improves transparency and keeps users informed about the progress of their orders.

- **Previous Orders and Refill Interface**

The previous orders interface allows users to view their past orders and review medicine details, pharmacy information, and order status. This feature helps users keep track of their medication history and previous purchases.

The refill interface allows users to reorder previously purchased medicines easily without searching for them again. This supports users who need recurring medicines and improves treatment continuity by simplifying repeated ordering.

- **Saved Medicines Interface**

The saved medicines interface enables users to store medicines they frequently use or want to access later. Saved medicines are displayed in an organized list or card layout, allowing users to quickly view medicine details, check availability, or add items to the cart.

This interface improves usability by reducing repeated search effort and supporting personal medication management. It is especially useful for users who take regular medicines or want to compare availability later.

- **Chat with Pharmacist Interface**

The chat interface allows users to communicate directly with pharmacists through the application. Users can ask questions about medicine availability, usage instructions, alternatives, or prescription-related issues. Messages are displayed in a conversational layout that supports clear communication between the user and pharmacist.

This interface supports one of the key objectives of MediSmart, which is improving interaction between users and pharmacists. It also enhances user confidence by providing access to professional guidance when needed.

- **Notifications Interface**

The notifications interface displays system alerts and updates for the user. These notifications may include order status changes, pharmacist responses, prescription review results, reminders, or important system messages.

The interface helps users stay informed about important activities without needing to manually check every section of the system. Notifications are displayed in a structured format with titles, descriptions, dates, and read/unread status.

- **User Profile and Address Management Interface**

The user profile interface allows customers to view and update their personal information, such as full name, phone number, email, preferred language, and account details. It provides a centralized area for managing user identity and account settings.

The address management interface allows users to add, edit, delete, or select default delivery addresses. This supports the home delivery order process by ensuring that correct address information is available when placing an order.

- **Pharmacist Dashboard Interface**

The pharmacist dashboard provides pharmacists with an overview of their pharmacy-related tasks. It displays new orders, pending prescription reviews, inventory alerts, and recent activities. This interface helps pharmacists manage daily operations efficiently.

The dashboard was designed with summary cards, tables, and action buttons to make important information easy to access. Through this interface, pharmacists can move quickly to order review, inventory management, or medicine update pages.

- **Pharmacist Order Review Interface**

The order review interface allows pharmacists to view customer orders, check ordered medicines, review uploaded prescriptions, and decide whether to confirm or reject the order. It displays order details, customer information, medicine quantities, and prescription attachments when available.

This interface supports the workflow shown in the pharmacist activity and sequence diagrams. It helps pharmacists make accurate decisions and update order status based on medicine availability and prescription validity.

- **Inventory Management Interface**

The inventory management interface allows pharmacists to manage medicines available in their pharmacy. Pharmacists can view medicine stock, update quantities, change prices, check expiry dates, and update stock status.

This interface is important for maintaining real-time availability information in the system. Since users depend on updated stock and price data, the inventory interface helps ensure that displayed information remains accurate and reliable.

- **Add and Update Medicine Interface**

The add and update medicine interface allows pharmacists to add new medicine records or update existing medicine information. The form includes fields such as medicine name, category, dosage form, strength, manufacturer, price, quantity, expiry date, usage instructions, and warnings.

This interface supports the pharmacist's role in keeping the medicine database complete and updated. It also helps improve search results and availability tracking for users.

- **Admin Dashboard Interface**

The admin dashboard provides administrators with a general overview of the system. It may include statistics about users, pharmacies, medicines, orders, and system activity. This interface helps administrators monitor the platform and access management functions quickly.

The dashboard was designed to support clear decision-making by displaying important information in cards, charts, or tables. It acts as the main control panel for system administration.

- **Manage Users Interface**

The manage users interface allows administrators to view, add, update, activate, deactivate, or delete user accounts. It displays user information such as name, email, phone number, role, and account status.

This interface supports role management by allowing the administrator to control access for customers, pharmacists, and admins. It also helps maintain system security and organization.

- **Manage Pharmacies Interface**

The manage pharmacies interface allows administrators to add and update pharmacy information, including pharmacy name, phone number, WhatsApp contact, email, city, district, address, working hours, and delivery availability.

This interface is important because pharmacy information is used in medicine availability, order placement, pharmacist assignment, and communication. Clear management of pharmacy records helps improve the accuracy of the platform.

- **Manage Categories Interface**

The manage categories interface allows administrators to create, edit, or remove medicine categories. These categories help organize medicines and improve the search and browsing experience for users.

The interface uses simple forms and tables to make category management clear and efficient. Organized categories also support better medicine classification and easier navigation.

- **Reports Interface**

The reports interface allows administrators to view system summaries and generate reports related to users, medicines, pharmacies, orders, and system activity. These reports support monitoring and evaluation of the platform's performance.

4.4 Backend Development

Backend development represents the core functional layer of the MediSmart system, as it is responsible for processing data, applying business logic, managing requests, and ensuring secure communication between the frontend interfaces and the database. In this project, the backend was implemented using a modern architecture based on Supabase as a Backend-as-a-Service platform, integrated with Next.js through Supabase Client APIs. Supabase was selected because it provides PostgreSQL database management, authentication, storage, real-time updates, and automatic API access within one integrated environment. This makes it suitable for the MediSmart system, which depends on real-time medicine availability, secure user access, prescription upload, order processing, pharmacist communication, and admin management.

4.4.1 Application Architecture

The backend architecture follows a modern service-based structure rather than a traditional MVC model. The system is divided into connected layers that work together to support system operations. The frontend layer is built with Next.js and React and is responsible for displaying interfaces and collecting user input. The service layer handles requests from the frontend, validates data, and sends operations to Supabase. The Supabase backend layer manages authentication, database queries, file storage, and real-time updates. Finally, the PostgreSQL database layer stores the main system entities such as users, pharmacies, medicines, pharmacy inventory, carts, orders, prescriptions, chats, and notifications, according to the ERD structure of the system.

This architecture improves maintainability and scalability because each part of the system has a clear responsibility. It also allows the system to expand in the future by adding new services, such as advanced medicine recommendations, payment integration, delivery tracking, or automated refill reminders.

4.4.2 Routing and Request Handling

Request handling in the MediSmart system is managed through the connection between Next.js pages/components and Supabase services. When a user performs an action, such as searching for a medicine, adding an item to the cart, uploading a prescription, or sending a chat message, the frontend sends the request to the backend layer using Supabase Client APIs or Next.js server-side functions.

Routes and requests are organized according to system roles and functions, including:

- Public requests, such as viewing active medicines and pharmacies.
- Customer requests, such as managing cart items, placing orders, uploading prescriptions, saving medicines, and viewing notifications.
- Pharmacist requests, such as reviewing orders, updating prescription status, managing inventory, and responding to chat messages.
- Admin requests, such as managing users, pharmacies, medicine categories, orders, and reports.

This organization ensures that each request is handled according to the user role and the required system operation.

4.4.3 Authentication and Authorization

Authentication was implemented using Supabase Auth, which manages user registration, login, email verification, session handling, and secure access. When users register or log in, Supabase verifies their credentials and creates a secure session. After authentication, the system checks the user role stored in the users table to determine the correct access level.

Authorization is applied through role-based access control. Customers can access medicine search, cart, order, prescription, chat, and profile features. Pharmacists can access pharmacy-related orders, prescriptions, inventory, and chat features. Administrators can access system management pages, including users, pharmacies, categories, reports, and order monitoring. This separation helps prevent unauthorized access and ensures that each actor can only perform the actions allowed for their role.

4.4.4 Database Interaction and Supabase Client API

The backend interacts with the PostgreSQL database through Supabase Client APIs. Each main entity in the system is represented by a database table, including users, pharmacies, medicines, pharmacy_inventory, carts, orders, prescriptions, chats, chat_messages, notifications, and saved_medicines. The backend performs operations such as inserting new records, updating existing records, deleting data when needed, and retrieving information for display in the frontend.

Supabase simplifies database interaction by providing direct and secure API access to tables while maintaining relational database integrity. Foreign key relationships ensure that connected records remain consistent, such as linking orders to users and pharmacies, linking inventory items to medicines and pharmacies, and linking prescription records to users and orders. This supports reliable data flow across the whole system.

4.4.5 Medicine Search and Inventory Module

The medicine search and inventory module is one of the most important backend components of the MediSmart system. It allows users to search for medicines by name, category, generic name, or keyword. The backend retrieves matching records from the medicines table and connects them with the pharmacy_inventory and pharmacies tables to display availability, quantity, price, and pharmacy details.

This module supports real-time medicine availability checking by retrieving the latest inventory information from the database. When pharmacists update stock quantity, price, or availability status, the updated information becomes available for users. This ensures that users can compare prices and availability across pharmacies before placing an order.

4.4.6 Cart, Order, and Prescription Module

The cart and order module manages the full ordering workflow. Users can add medicines to the cart, update quantities, remove items, and confirm their order. When an order is placed, the backend creates a new order record, stores ordered items in the order_items table, links the order to the selected pharmacy, and assigns the correct order status.

For medicines that require a prescription, the backend supports prescription upload through Supabase Storage. The prescription file is stored securely, while its file URL and related information are saved in the prescriptions table. The prescription is then linked to the order so that the pharmacist can review it before confirming or rejecting the request. This module supports safe and organized prescription handling within the system.

- **Backend Code Sample for Order and Prescription Processing**

```
async function uploadPrescription() {
  if (!file || !profile) return null;

  const filePath = `${profile.id}/${Date.now()}-${file.name}`;

  const { error: uploadError } = await supabase.storage
    .from('prescriptions')
    .upload(filePath, file);

  if (uploadError) throw uploadError;

  const { data: publicData } = supabase.storage
    .from('prescriptions')
    .getPublicUrl(filePath);

  const { data, error } = await supabase
    .from('prescriptions')
    .insert({
```

```
      user_id: profile.id,
      image_url: publicData.publicUrl,
      uploaded_at: new Date().toISOString(),
      notes: notes
    })
    .select('id')
    .single();

  if (error) throw error;

  return data.id;
}

async function placeOrder() {
  if (!profile || !cart || !items.length) return;

  if (orderType === 'home_delivery' && !addressId) {
    setMessage('Please choose a delivery address.');
    return;
  }

  setPlacing(true);

  try {
    const prescriptionId = await uploadPrescription();

    const { data: order, error } = await supabase
      .from('orders')
      .insert({
        order_number: generateOrderNumber(),
        user_id: profile.id,
        pharmacy_id: cart.pharmacy_id,
        address_id: orderType === 'home_delivery' ? addressId : null,
        prescription_id: prescriptionId,
        order_type: orderType,
        status: 'pending',
        notes: notes
      })
      .select('*')
      .single();

    if (error) throw error;

    await supabase.from('order_items').insert(
      items.map((item) => ({
        order_id: order.id,
        medicine_id: item.medicine_id,
        quantity: item.quantity,
        unit_price: item.unit_price,
        total_price: Number(item.unit_price) * item.quantity
      }))
```

```
    );

    await supabase.from('notifications').insert({
      user_id: profile.id,
      title: 'Order submitted',
      body: `Your order ${order.order_number} is pending pharmacist review.`,
      type: 'order'
    });

    await supabase
      .from('cart_items')
      .delete()
      .eq('cart_id', cart.id);

    setMessage(`Order ${order.order_number} placed successfully.`);
  } catch (error: any) {
    setMessage(error.message || 'Could not place order.');
  } finally {
    setPlacing(false);
  }
}
```

This code demonstrates the backend processing of the order workflow. It uploads the prescription file to Supabase Storage, creates a prescription record, inserts the order into the orders table, stores ordered medicines in the order_items table, sends a notification to the user, and clears the cart after successful order placement. This sample reflects the backend role in managing order creation, prescription handling, notification generation, and database updates.

4.4.7 Pharmacist Management Module

The pharmacist module supports pharmacy-side operations. Pharmacists can view incoming orders, review prescription attachments, confirm or cancel orders, update order status, and manage pharmacy inventory. When a pharmacist confirms an order, the backend updates the order status and deducts the ordered quantity from the pharmacy inventory.

The module also allows pharmacists to add or update medicine information, including quantity, price, expiry date, and stock status. These updates are reflected in the medicine search and availability interfaces, helping users access accurate information. This module is directly connected to the pharmacist activity and sequence diagrams because it represents the main workflow of pharmacist responsibilities.

4.4.8 Chat and Notification Module

The chat module enables communication between users and pharmacists through the system. When a user sends a message, the backend stores it in the chat_messages table and connects it to the related

chat record. Pharmacists can reply through their dashboard, and the conversation remains stored for future reference.

The notification module informs users and pharmacists about important events, such as new orders, prescription review results, order status changes, and new chat messages. Notifications are stored in the notifications table and displayed in the user interface. This improves communication and keeps users updated without requiring them to manually check every section of the system.

4.4.9 Admin Management Module

The admin management module provides backend support for system administration. Administrators can manage users, pharmacies, categories, medicines, and orders. Admin actions include adding or updating pharmacy records, managing user accounts, controlling medicine categories, monitoring order activity, and generating reports.

Each admin operation is protected by role-based authorization to ensure that only administrators can perform management actions. This module helps maintain system organization, data accuracy, and overall platform control.

4.4.10 Security and Data Protection

Security was considered throughout backend development to protect user information, prescription data, order details, and pharmacy records. Supabase Auth provides secure authentication and session management, while role-based access control restricts system functions according to user type. Input validation is used to reduce incorrect data entry, and database constraints help maintain data accuracy.

Supabase Storage is used to manage prescription files securely, while database relationships and permissions help prevent unauthorized access to sensitive records. These security measures support the non-functional requirements of the system, especially reliability, privacy, and data protection.

4.5 Conclusion

In summary, this chapter presented the implementation of the MediSmart system by explaining the database, frontend interfaces, and backend logic. The backend was developed using a modern architecture based on Supabase and Next.js integration, allowing the system to support authentication, real-time database operations, prescription storage, medicine availability tracking, order management, chat communication, notifications, pharmacist operations, and admin management. Through this integrated implementation, MediSmart becomes a functional and reliable smart pharmacy platform that supports users, pharmacists, and administrators while meeting the functional and non-functional requirements defined in the previous chapters.

Chapter 5: System Testing & Deployment

5.1 Introduction

System testing and deployment represent important stages in the software development life cycle, as they ensure that the implemented system works correctly and is ready for real use. In this chapter, the MediSmart smart pharmacy system was tested to verify that its main functions meet the requirements defined in the previous chapters. The testing process focused on checking the main workflows for customers, pharmacists, and administrators, including registration, login, medicine search, availability checking, cart and order management, prescription upload, pharmacist order review, inventory management, chat, notifications, and admin management. After completing the testing process, the system was prepared for deployment using InfinityFree hosting for the web interface, while Supabase continued to manage backend services such as the PostgreSQL database, authentication, storage, and real-time data operations.

5.2 Testing Objectives

The testing objectives define the main goals of evaluating the MediSmart system before deployment. The purpose of testing is to ensure that the system works correctly, securely, and efficiently for all users, including customers, pharmacists, and administrators. The main testing objectives are:

1. To verify that users can register, log in, and access the system securely.
2. To ensure that role-based access works correctly for customers, pharmacists, and administrators.
3. To confirm that users can search for medicines by name, category, generic name, or keyword.
4. To verify that medicine availability, stock quantity, pharmacy information, and prices are displayed correctly.
5. To ensure that users can add medicines to the cart, update quantities, remove items, and place orders successfully.
6. To test the prescription upload function and confirm that prescription files are stored and linked to orders correctly.
7. To verify that pharmacists can review orders, check prescriptions, confirm or reject orders, and update order status.
8. To ensure that pharmacists can add, update, and manage medicine inventory information.
9. To confirm that administrators can manage users, pharmacies, medicine categories, orders, and reports.

10. To test the chat and notification features to ensure effective communication between users and pharmacists.
11. To check that data is saved, updated, retrieved, and displayed correctly through Supabase.
12. To evaluate the usability and responsiveness of the system interfaces on different screen sizes.

5.3 Testing Methodologies

Several testing methodologies were applied to evaluate the MediSmart system and ensure that it meets the functional and non-functional requirements defined in the previous chapters. The testing methodologies used in this project are:

1. **Functional Testing**

   Functional testing was used to verify that each feature performs its expected task correctly. This included testing registration, login, medicine search, cart management, order placement, prescription upload, inventory update, chat, notifications, and admin management.

2. **Integration Testing**

   Integration testing was performed to ensure that the frontend interfaces are correctly connected with Supabase services. This included checking the connection between Next.js pages and the Supabase database, authentication, storage, and API operations.

3. **User Interface Testing**

   User interface testing was used to evaluate the clarity, organization, and usability of the system pages. The interfaces were checked to ensure that buttons, forms, tables, cards, navigation menus, and messages appear correctly and are easy to use.

4. **Role-Based Access Testing**

   Role-based access testing was applied to confirm that each user type can only access the features assigned to them. Customers were tested on medicine search and order functions, pharmacists on order and inventory functions, and administrators on management and reporting functions.

5. **Database Testing**

   Database testing was used to check that data is inserted, updated, deleted, and retrieved correctly from Supabase PostgreSQL tables. It also verified that relationships between users, medicines, pharmacies, inventory, orders, prescriptions, chats, and notifications work correctly.

6. **Security Testing**

   Security testing focused on protecting user data, prescription records, and order information. It included checking authentication, protected pages, input validation, user sessions, and role restrictions.

7. **Responsiveness Testing**

   Responsiveness testing was performed to ensure that the system works properly on different screen sizes, including desktop, tablet, and mobile views. This helped confirm that the interface remains readable and usable across devices.

8. **Deployment Testing**

   Deployment testing was conducted after uploading the frontend files to InfinityFree hosting. The purpose was to confirm that the deployed system loads correctly online and can still communicate with Supabase services for authentication, database operations, and storage.

5.4 Testing Tools

The testing process used several tools to support system evaluation and debugging. Visual Studio Code was used for reviewing and editing the system code during development. Browser developer tools were used to inspect interface behavior, check console errors, test responsiveness, and verify page performance. The Supabase Dashboard was used to monitor authentication users, database records, storage files, and table relationships. The Supabase SQL Editor was used to run queries and validate database operations. In addition, the local Next.js development server was used to test the system before deployment, while InfinityFree hosting was used to test the system in an online environment.

5.5 Testing Process

The testing process began by preparing the development environment and confirming that all required environment variables were correctly connected to the Supabase project. After that, authentication functions were tested by creating accounts, logging in, and checking that users were redirected to the correct dashboard according to their roles. The customer workflow was then tested by searching for medicines, filtering results, checking pharmacy availability, adding items to the cart, uploading prescriptions, placing orders, and viewing notifications. The pharmacist workflow was tested by reviewing incoming orders, checking prescription attachments, accepting or rejecting orders, updating order status, and managing inventory quantities and prices. The administrator workflow was tested by managing users, pharmacies, categories, and reports.

During testing, database changes were checked through Supabase to ensure that each action created or updated the correct records. For example, when a user placed an order, the order information was

checked in the orders table, the ordered medicines were checked in the order_items table, and the notification was checked in the notifications table. The prescription upload function was also verified by checking that the uploaded file was stored correctly and linked to the related order. Finally, the system interfaces were tested on different screen sizes to confirm that the design remained clear, responsive, and easy to use.

5.6 Testing Outcomes

The testing results showed that the MediSmart system was able to perform its main functions successfully. Users were able to register, log in, search for medicines, compare prices, add medicines to the cart, upload prescriptions, and place orders. Pharmacists were able to view orders, review prescriptions, update order status, and manage inventory information. Administrators were able to access management pages and control users, pharmacies, categories, and reports. The integration with Supabase worked properly for authentication, database operations, and file storage. The interface was also clear and responsive, allowing users to navigate between pages smoothly.

The testing phase also helped identify areas that can be improved in future versions. These improvements may include adding online payment, delivery tracking, advanced medicine recommendations, pharmacist availability status, and more detailed analytical reports. Overall, the testing outcomes indicate that the system meets its main functional and non-functional requirements and is ready for deployment.

5.7 System Deployment

System deployment is the process of publishing the implemented system so that it can be accessed by users through the internet. In this project, InfinityFree hosting was selected to deploy the web interface of the MediSmart system. InfinityFree supports uploading custom websites using FTP or an online File Manager, and it provides support for PHP and MySQL hosting; however, free InfinityFree hosting does not support running Node.js applications directly.

Because the MediSmart frontend was developed using Next.js, the system should be prepared as a static exported website before uploading it to InfinityFree. Next.js static export allows the project to be hosted on any web server that can serve HTML, CSS, and JavaScript static assets, and the generated production files are placed in the out folder after building the project. In this deployment approach, InfinityFree hosts the exported frontend files, while Supabase continues to provide backend services such as authentication, PostgreSQL database access, prescription storage, and real-time data updates.

The deployment process includes the following steps:

1. Create and configure the Supabase project.
2. Run the database schema in the Supabase SQL Editor.

3. Create the required storage bucket for prescription files.
4. Configure Supabase authentication settings.
5. Add the Supabase project URL and anonymous key to the frontend environment file.
6. Prepare the Next.js project for production build.
7. Export the frontend as static files.
8. Upload the exported files to the InfinityFree htdocs directory using the File Manager or FTP.
9. Test the website link after uploading the files.
10. Verify that login, medicine search, cart, order placement, prescription upload, and dashboards communicate correctly with Supabase services.

After deployment, the system was checked online to ensure that the main pages loaded correctly and that the frontend could still connect to Supabase. The testing confirmed that the deployed version was able to display medicine data, process user actions, and support the main system workflows.

5.8 Conclusion

This chapter presented the testing and deployment of the MediSmart smart pharmacy system. The testing phase verified that the main system functions operated correctly across customer, pharmacist, and administrator workflows. It also confirmed that the system interfaces were clear and responsive, the database operations were accurate, and the integration with Supabase services was successful. The deployment section explained how the system can be published using InfinityFree hosting for the frontend while relying on Supabase for backend services. Overall, the testing and deployment results show that MediSmart is ready to operate as a functional web-based smart pharmacy platform that supports medicine discovery, real-time availability checking, prescription management, order processing, communication, and system administration.

Chapter 6: Conclusion and Future Works

6.1 Conclusion

In conclusion, this case study presented the analysis, design, implementation, testing, and deployment of the MediSmart smart pharmacy system, a web-based platform proposed to improve access to pharmaceutical services and support efficient medicine discovery and order management. The case study addressed challenges such as difficulty finding available medicines, limited price transparency, repeated visits to pharmacies, and the need for easier communication with pharmacists. The system requirements were identified and translated into structured models, including use case diagrams, activity diagrams, ERD, DFD, and sequence diagrams, which clarified workflows, data relationships, and user interactions. The implementation demonstrated how the proposed system can be developed using Next.js and React for the frontend and Supabase for backend services, including PostgreSQL, authentication, storage, and real-time data processing. The testing and deployment phase verified the main workflows and explained how the system can be published using InfinityFree hosting while relying on Supabase for backend operations. Overall, this case study shows that a smart pharmacy platform can improve medicine accessibility, reduce time and effort, enhance communication between users and pharmacists, and support digital transformation in healthcare services.

6.2 Future Works

1. The system will be enhanced by adding electronic payment options such as Mada, credit cards, and Apple Pay, in addition to cash payment. This will make the ordering process more flexible and provide users with safer and more convenient payment choices.
2. A loyalty points system will be added to allow users to collect points for every purchase made through the platform. These points will later be converted into discounts, which encourages repeated use of the system and improves customer satisfaction.
3. The system will be connected with delivery service providers to improve medicine delivery operations. This will support faster delivery, clearer order tracking, and better coordination between pharmacies, delivery representatives, and customers.
4. Barcode and QR code scanning will be added to speed up medicine identification, reduce manual entry errors, and improve inventory and sales accuracy, especially when pharmacists add, update, or dispense medicines.
5. A dedicated mobile application will be developed for Android and iOS to improve accessibility and usability. The application will allow users to search for medicines, place

orders, upload prescriptions, receive notifications, and communicate with pharmacists more easily.

6. The system will be expanded to support the management of multiple pharmacy branches through a single platform. This will allow pharmacy owners or administrators to monitor stock levels, medicine availability, and order activity across different branches.
7. The system will include advanced analytical reports that help administrators and pharmacy owners monitor sales, most requested medicines, low-stock items, user activity, and pharmacy performance. This will support better decision-making and improve inventory planning.
8. The system will add automatic medicine reminders and refill alerts to notify users about medication schedules and repeated medicine needs. This will support treatment continuity and help users manage recurring medicines more effectively.

References

[1] P. A. Khan, P. J. Badole, and D. Shaikh, "Telemedicine and digital health: The future for pharmaceutical companies and health care," *Indian J. Pharmacol.*, vol. 58, no. 1, pp. 24–34, 2026.

[2] A. Bandyopadhyay et al., "Digital Health Platforms in Pharmacy: Transforming Patient Engagement and Care," *Current Pharmaceutical Design*, vol. 32, 2026. doi: 10.2174/0113816128440451260123110515.

[3] Aiswarya et al., "Medic – The Medicine Availability Checker," *Zenodo*, 2025. Available: https://zenodo.org/records/15541961

[4] K. Kommey et al., "MeDASH: An Effective Way to Check the Availability of Pharmaceutical Drugs in Proximity Using QR-codes," *Spekta*, vol. 5, no. 2, pp. 144–167, 2024. doi: 10.12928/spekta.v5i2.8994.

[5] N. M. Shapri et al., "SmartMY: An Easy-access Medication Price Checker Mobile Application Prototype," *Int. J. Pharm. Nutr. Cosmet. Sci.*, vol. 8, Suppl. 1, pp. 26–29, 2025. doi: 10.24191/IJPNaCS.v8suppl1.07.

[6] R. Agudelo, J. Granados, M. Ceballos, and J. A. Pereañez, "Design and Development of a Mobile Application for Drug Information and Other Health Data for Users and Patients of Pharmacies and Outpatient Pharmaceutical Services," *Explor. Res. Clin. Soc. Pharm.*, vol. 20, p. 100661, 2025.

[7] "MEDISYNC: A Smart Medicine Tracking and Pharmacy Integration System," *Int. J. Emerg. Technol. Innov. Res.*, vol. 12, no. 7, pp. 868–873, 2025. Available: http://www.jetir.org/papers/JETIRGX06165.pdf.

[8] IBM, "What is the Software Development Lifecycle (SDLC)?," *IBM Think*, 2023. Available: https://www.ibm.com/think/topics/sdlc

[9] Microsoft, "What is the Software Development Life Cycle?," *Microsoft Learn*, 2023. Available: https://learn.microsoft.com/

[10] J. Santos and E. Dano, "Systems Analysis," *Encyclopedia of Sustainable Technologies*, 2nd ed., Elsevier, 2024. Available: https://www.sciencedirect.com/topics/computer-science/systems-analysis

[11] S. W. Ambler, *The Elements of UML™ 2.0 Style*, Cambridge University Press, 2005. Available: https://www.cambridge.org/core/books/elements-of-uml-20-style

[12] L. Baresi and M. Pezzè, "Behavioral Modeling with Activity Diagrams," *Encyclopedia of Database Systems*, Springer, 2009. Available: https://link.springer.com/referenceworkentry/10.1007/978-0-387-39940-9_9

[13] GeeksforGeeks, "System Design Introduction – LLD & HLD," 2026. Available: https://www.geeksforgeeks.org/system-design/getting-started-with-system-design/

[14] Spec Innovations, "Class Diagram," 2025. Available: https://help.specinnovations.com/class-diagram

[15] ScienceDirect, "Data Flow Diagram," 2025. Available: https://www.sciencedirect.com/topics/computer-science/data-flow-diagram

[16] Sparx Systems, "UML 2 Tutorial – Sequence Diagram," 2023. Available: https://sparxsystems.com/resources/tutorials/uml2/sequence-diagram.html

الملخص

تقدم هذه الدراسة منصة صيدلية ذكية تتمحور حول المستخدم، وتهدف إلى معالجة التحديات المرتبطة بالحصول على الأدوية. يوفر النظام بيئة رقمية موحدة تربط بين عدة صيدليات، مما يتيح للمستخدمين البحث عن الأدوية، ومقارنة الأسعار، والتحقق من توفرها في الوقت الحقيقي ضمن المواقع القريبة. بالإضافة إلى ذلك، يدمج النظام وحدة تواصل تدعم التفاعل المباشر بين المستخدمين والصيادلة من خلال نظام دردشة داخل التطبيق لتقديم الاستشارات والإرشادات. وفي الحالات التي تتطلب دعماً إضافياً، يتيح النظام التصعيد إلى قنوات تواصل خارجية مثل تطبيق واتساب باستخدام بيانات الاتصال المسجلة للصيدلية، بما يضمن استمرارية الرعاية الصحية. كما يدعم النظام ميزة الحفظ الذكي لسجل الأدوية وإدارة الوصفات الطبية، مما يتيح إعادة طلب الأدوية المتكررة بكفاءة ويسهم في استمرارية العلاج. تم تطوير النظام باستخدام تقنيات ويب حديثة، حيث استُخدم Next.js لبناء الواجهة الأمامية، وSupabase لتقديم خدمات الواجهة الخلفية التي تشمل إدارة قاعدة البيانات، والمصادقة، ومعالجة البيانات في الوقت الحقيقي، بما يحقق قابلية التوسع والكفاءة. ويستند النظام إلى نموذج قاعدة بيانات منظم وسير عمل واضح يضمن إدارة فعالة للمستخدمين والصيدليات والأدوية والطلبات وعمليات التواصل. وتبرز هذه الدراسة أهمية الأنظمة الصيدلية الذكية في تحسين الوصول إلى الأدوية، وتعزيز تجربة المستخدم، ودعم التحول الرقمي في قطاع الرعاية الصحية.

الكلمات المفتاحية: نظام صيدلية ذكي، اكتشاف الأدوية، التوفر في الوقت الحقيقي، التفاعل مع الصيدلي، إدارة الوصفات الطبية، الرعاية المتمحورة حول المريض.